

Claude Code 完全実践ガイド

スキル・フック・メモリ・MCP を使いこなす

2026-06-14

- Claude Code 完全実践ガイド
 - スキル・フック・メモリ・MCP を使いこなす
 - この教材の使い方（3ステップ）
 - カテゴリー一覧
 - 推奨学習ルート
 - 本書で学べること
 - 対象読者
- 0. 基礎と環境準備
 - 0.1 0-1: Claude Code とは
 - 0.2 0-2: インストールとセットアップ
- 1. コア機能
 - 1.1 1-1: チャットとファイル編集
 - 1.2 1-2: スラッシュコマンド一覧
 - 1.3 1-3: ターミナル・Bash ツール
 - 1.4 1-4: コンテキスト管理
- 2. スキルシステム
 - 2.1 2-1: スキルとは
 - 2.2 2-2: /skill-creator ハンズオン
 - 2.3 2-3: 組み込みスキル活用
 - 2.4 2-4: カスタムスキル作成
- 3. フックと自動化
 - 3.1 3-1: フックの概要

- 3.2 3-2: フックの種類
- 3.3 3-3: 自動化パターン
- 3.4 3-4: settings.json 設定
- 4. メモリシステム
 - 4.1 4-1: メモリの種類
 - 4.2 4-2: CLAUDE.md ファイル
 - 4.3 4-3: 永続的なメモリ（オートメモリ）
 - 4.4 4-4: メモリのベストプラクティス
- 5. MCP とエージェント
 - 5.1 5-1: MCP の概要
 - 5.2 5-2: MCP サーバー設定
 - 5.3 5-3: エージェント SDK
 - 5.4 5-4: マルチエージェント
 - 5.5 5-5: カスタムエージェント実装
- 6. 発展・応用
 - 6.1 6-1: CI/CD 統合
 - 6.2 6-2: チームワークフロー
 - 6.3 6-3: パフォーマンス Tips
 - 6.4 6-4: トラブルシューティング

Claude Code 完全実践ガイド

スキル・フック・メモリ・MCP を使いこなす

この教材は、Anthropic の AI コーディングアシスタント **Claude Code** を基礎から応用まで体系的に学ぶ実践ガイドです。スキルシステム・フック自動化・メモリ管理・MCP/エージェント連携の4大機能を完全カバーします。

💡 ブラウザで <https://duwenji.github.io/spa-quiz-app/> を開くと、関連トピックをクイズ形式で復習できます。

 全体ガイド | 前提: Claude Code インストール済み

🔔 **更新状況:**  Claude Code 最新バージョン対応中

最終更新: 2026年6月 | スキル・フック・MCP 1.0 対応

この教材の使い方（3ステップ）

1. Part 0 で環境を整えて Claude Code を起動する
2. Part 1〜2 でコア機能とスキルシステムを体験する
3. Part 3〜6 で自動化・メモリ・エージェント連携まで習得する

カテゴリー一覧

#	Part	難易度	学習時間
0	基礎と環境準備	★	20分
1	コア機能	★ ★	60分
2	スキルシステム	★ ★	80分
3	フックと自動化	★ ★	60分
4	メモリシステム	★ ★	60分
5	MCP とエージェント	★ ★ ★	90分
6	発展・応用	★ ★ ★	65分

推奨学習ルート

- 最短で体験する: 0 → 1 → 2
- 自動化を極める: 3 → 4
- エージェント開発: 5 → 6

本書で学べること

- Claude Code の設計思想とアーキテクチャを理解できる
- スキルシステムでチームの作業を自動化できる
- フックで繰り返し作業をゼロにできる
- CLAUDE.md と永続メモリでコンテキストを管理できる

- MCP サーバーで外部ツールと連携できる
- カスタムエージェントを設計・実装できる

対象読者

- Claude Code を使い始めた開発者
 - スキルやフックで業務効率を上げたいエンジニア
 - AI コーディングアシスタントの全機能を体系的に習得したいチーム
-

0. 基礎と環境準備

0.1 0-1: Claude Code とは

学習時間: 10分 | 難易度: ★

0.1.1 概要

Claude Code は Anthropic が開発した AI コーディングアシスタントです。ターミナル上で動作する CLI ツールとして設計されており、コードの生成・編集・レビュー・デバッグをインタラクティブに行えます。

Claude Code は単なるチャットボットではなく、**実際にファイルを読み書きし、コマンドを実行できるエージェント**です。

0.1.2 設計思想

Claude Code の設計は3つの原則に基づいています。

1. エージェント的に動作する

ファイルの読み書き、ターミナルコマンドの実行、外部 API の呼び出しなど、実際に作業を完遂するために必要なアクションを自律的に取ります。

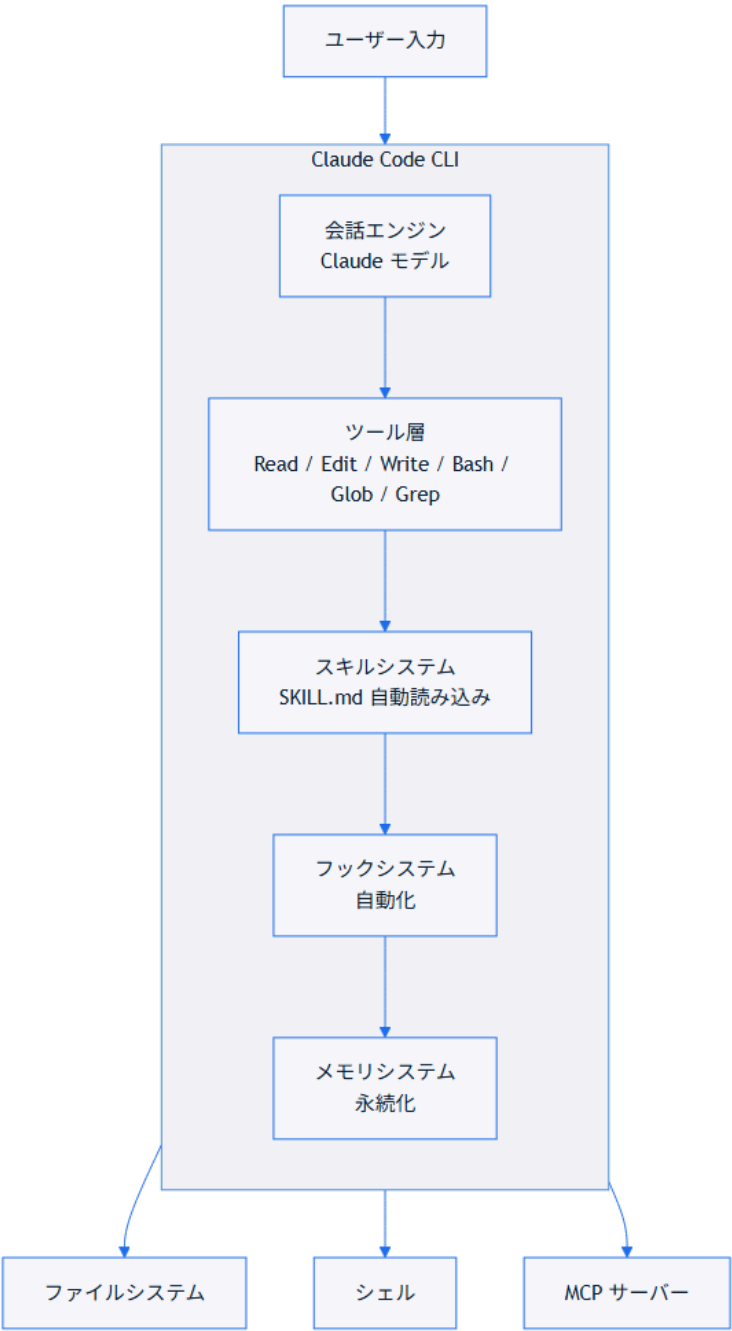
2. 人間の監督を前提とする

破壊的な操作（ファイル削除、force push など）は事前に確認を求めます。ユーザーが最終的な意思決定者です。

3. プロジェクトに馴染む

CLAUDE.md やメモリシステムを通じて、プロジェクト固有のルールや過去の文脈を記憶します。

0.1.3 アーキテクチャ



0.1.3.1 Windows でのコマンド実行

アーキテクチャ図の「Bash」はツール名であり、実際に使われるシェルは OS によって異なります。

環境	主要シェル	Bash ツール
macOS / Linux	bash / zsh	同じシェルを使用
Windows	PowerShell 7+	Git Bash（Git for Windows が必要）

Windows では **PowerShell** が**主要シェル**として動作するため、`git`・`npm`・`docker` などほぼすべての操作を PowerShell 経由で実行できます。Bash ツールは POSIX 系スクリプトを実行したい場合に補助的に使います（パス区切り文字はスラッシュ `/` が必要）。

0.1.4 他ツールとの比較

観点	Claude Code	GitHub Copilot	Cursor
動作場所	ターミナル / IDE 拡張	IDE 拡張	IDE（独自）
エージェント機能	フル（ファイル操作・コマンド実行）	限定的	限定的
スキルシステム	あり（SKILL.md）	なし	なし
フック・自動化	あり（hooks）	なし	なし
メモリ永続化	あり（ファイルベース）	なし	なし
MCP 対応	あり	なし	なし
オープンソース	CLI はオープン	クローズド	クローズド

0.1.5 利用できる環境

- **CLI（ターミナル）**：ネイティブインストーラーまたは npm でインストール（詳細は次章）
- **VS Code 拡張**: Marketplace から「Claude Code」を検索（Cursor でも利用可能）

- **JetBrains プラグイン:** JetBrains Marketplace から入手（IntelliJ IDEA・PyCharm・WebStorm 等）
- **Desktop App:** Mac（Intel / Apple Silicon）・Windows（x64 / ARM64）対応
- **Web:** claude.ai/code でブラウザから利用（ローカルセットアップ不要）
- **iOS アプリ:** Claude iOS アプリ経由でクラウドセッションを開始・継続可能
- **CI/CD:** GitHub Actions・GitLab CI/CD と連携してコードレビューを自動化
- **Slack:** [@Claude](#) にメンションしてタスクをルーティング可能

0.1.6 Claude Code が得意なこと

タスク	説明
コード生成	要件から実装コードを生成
リファクタリング	既存コードの改善・整理
デバッグ	エラーの原因特定と修正
コードレビュー	品質・セキュリティ・パフォーマンスの評価
テスト生成	ユニットテスト・統合テストの自動生成
ドキュメント作成	コメント・README の生成
Git 操作	コミット・PR 作成の補助
マルチエージェント	複数のサブエージェントを生成して並行作業
定期タスク自動化	スケジュール実行（ルーティン）で繰り返し作業を自動化
MCP 連携	Google Drive・Jira・Slack などの外部ツールと接続

0.1.7 🏆 練習問題

1. **【確認】** Claude Code が「単なるチャットボット」と異なる点を2つ挙げてください。
2. **【確認】** Claude Code の設計原則3つを説明してください。

3. 【実践】 Claude Code と GitHub Copilot の違いを、自分のユースケースに当てはめて考えてみましょう。どの機能が最も役立ちそうですか？
4. 【応用】 あなたが日常のコーディング作業でよく行うタスク（デバッグ、コードレビューなど）を1つ選び、Claude Code がどう支援できるか想像してみましょう。

0.1.8 次のステップ

→ 0-2: インストールとセットアップ に進む

0.2 0-2: インストールとセットアップ

学習時間: 10分 | 難易度: ★

0.2.1 前提条件

0.2.1.1 システム要件

項目	要件
OS	macOS 13.0+、Windows 10 1809+、Ubuntu 20.04+、Debian 10+、Alpine 3.19+
RAM	4GB 以上
ネットワーク	インターネット接続必須
Node.js	npm 経由でインストールする場合のみ 18 以上が必要

0.2.1.2 アカウント要件

Claude.ai の有料プラン（Pro / Max / Team / Enterprise）または Anthropic Console アカウントが必要です。無料プランは Claude Code を利用できません。

0.2.2 インストール

0.2.2.1 推奨: ネイティブインストーラー

ネイティブインストーラーは Node.js 不要で、バックグラウンドで自動更新されます。

macOS / Linux / WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

0.2.2.2 その他のインストール方法

Homebrew (macOS) :

```
brew install --cask claude-code
```

`claude-code` は安定版チャンネル（約1週間遅れ）、`claude-code@latest` は最新チャンネルを追跡します。

WinGet (Windows) :

```
winget install Anthropic.ClaudeCode
```

Linux パッケージマネージャー (apt / dnf / apk) :

Debian/Ubuntu (apt) 、Fedora/RHEL (dnf) 、Alpine Linux (apk) でもリポジトリ経由でインストールできます。詳細は公式ドキュメントを参照してください。

npm (Node.js 18+ が必要) :

```
npm install -g @anthropic-ai/claude-code
```

更新方法の違い: ネイティブインストールはバックグラウンドで自動更新されます。Homebrew・WinGet・Linux パッケージマネージャー経由のインストールは自動更新されないため、手動で `brew upgrade claude-code` / `winget upgrade Anthropic.ClaudeCode` / `npm install -g @anthropic-ai/claude-code@latest` を実行して更新してください。npm インストールは Claude Code 内部で自動更新されますが、npm の書き込み権限がない場合は手動更新が必要です。

0.2.3 インストール確認

```
claude --version
```

より詳細な診断（インストール状態・認証・ネットワーク接続を確認）

```
claude doctor
```

手動で最新版に更新するには：

```
claude update
```

0.2.4 認証設定

0.2.4.1 方法1: Claude.ai アカウント（推奨・通常の使い方）

初回起動時に `claude` を実行するとブラウザが開き、Claude アカウントでサインインできます。

```
claude
```

→ 初回起動時に認証を求めるプロンプトが表示される

対応プランと特徴：

プラン	月額（目安）	Claude Code での利用
Max	\$100/\$200	レート制限が高く最適
Pro	\$20	利用可能（ヘビーな使用時に制限あり）
Team	\$25/人	チーム向け、管理機能あり
Enterprise	要相談	SSO・管理コンソール対応

注意: 無料の Claude.ai プランは Claude Code を利用できません。

その他の認証コマンド：

```
claude auth logout    # ログアウト
claude auth status    # 認証状態を確認
```

0.2.4.2 方法2: Anthropic Console / API キー・サードパーティプロバイダー

Amazon Bedrock・Google Vertex AI・Microsoft Foundry など外部プロバイダー経由、または API キーで利用する場合：

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

永続的に設定するには `~/ .bashrc` / `~/ .zshrc` に追加します。

注意: `ANTHROPIC_API_KEY` が設定されている場合、サブスクリプションよりAPIキーが優先されます。

0.2.5 初回起動

プロジェクトのルートディレクトリで起動します：

```
cd my-project
claude
```

初回起動時に認証方法の選択など、いくつかの設定確認が行われます。

0.2.6 IDE 拡張のインストール

0.2.6.1 VS Code

1. VS Code を開く
2. 拡張機能（Ctrl+Shift+X）を開く
3. 「Claude Code」を検索してインストール
4. コマンドパレット（Ctrl+Shift+P）から `Claude Code: Open` を実行

0.2.6.2 JetBrains

1. Settings → Plugins → Marketplace
2. 「Claude Code」を検索してインストール

0.2.7 CLAUDE.md の初期化

プロジェクトルートで `CLAUDE.md` を自動生成できます：

```
claude
/init
```

Claude Code がプロジェクトを分析し、CLAUDE.md の初稿を作成します。

0.2.8 設定ファイルの場所

ファイル	場所	用途
グローバル設定	<code>~/.claude/settings.json</code>	全プロジェクト共通設定
プロジェクト設定	<code>.claude/settings.json</code>	プロジェクト固有設定
プロジェクト指示	<code>CLAUDE.md</code>	プロジェクトのコンテキスト
個人用スキル	<code>~/.claude/skills/</code>	全プロジェクトで使えるスキル
プロジェクト用スキル	<code>.claude/skills/</code>	そのプロジェクト専用スキル
メモリ	<code>~/.claude/projects/<hash>/memory/</code>	プロジェクト別の永続メモリ
MCP 設定	<code>.mcp.json</code>	プロジェクトの MCP サーバー設定

0.2.9 モデルの選択

`settings.json` でモデルを指定できます：

```
{
  "model": "claude-sonnet-4-6"
}
```

利用可能なモデル（エイリアスまたはフル ID で指定）：

モデルエイリアス	フル ID 例	特徴
<code>opus</code>	<code>claude-opus-4-6</code>	最高性能、複雑なタスク向け
<code>sonnet</code>	<code>claude-sonnet-4-6</code>	バランス型（デフォルト）
<code>haiku</code>	<code>claude-haiku-4-5</code>	高速・軽量

エイリアス（`"opus"`・`"sonnet"`・`"haiku"`）を使うと、同じ世代の最新モデルへ自動的に追従されます。`/model` コマンドやセッション内で切り替えることもできます。

0.2.10 動作確認

```
# インストールと設定の診断
claude doctor

# 起動して動作確認
claude
> Hello! このプロジェクトの README.md を読んで概要を教えて
```

`claude doctor` はインストール状態・認証・ネットワーク接続などを診断し、問題があれば修正方法を提示します。Claude Code がファイルを読み込み、プロジェクトの概要を説明します。

0.2.11 🏆 練習問題

1. **【確認】** ネイティブインストーラーと npm インストールの違いを1つ教えてください。
2. **【実践】** `claude --version` と `claude doctor` を実行し、バージョンと診断結果を確認しましょう。
3. **【実践】** プロジェクトディレクトリで `claude` を起動し、`/init` で CLAUDE.md を生成してみましょう。
4. **【応用】** チームで Claude Code を導入する場合、API キー認証と claude.ai プラン認証のどちらが適切ですか？理由とともに教えてください。

0.2.12 次のステップ

→ 1-1: チャットとファイル編集 に進む

1. コア機能

1.1 1-1: チャットとファイル編集

学習時間: 15分 | 難易度: ★★

1.1.1 基本的な会話

Claude Code は対話形式で作業します。自然言語で指示を出すだけで、必要なツールを自律的に選択して実行します。

> このディレクトリにある TypeScript ファイルの一覧を教えてください

Claude: 確認します。

[Glob ツールを実行: **/*.ts]

```
src/  
├─ index.ts  
├─ components/Button.tsx  
└─ utils/helpers.ts
```

1.1.2 ファイル読み込み

Claude Code はプロジェクト内のファイルを直接読み込んで作業します。

> src/utils/helpers.ts を読んで、何をしているか説明して

明示的にファイルを指定しなくても、文脈から関連ファイルを自動的に読み込みます。

1.1.3 コード生成

- ＞ React コンポーネント `Button.tsx` を作成して。
- ＞ プロパティ: `label(string)`, `onClick(function)`, `disabled(boolean)`
- ＞ スタイル: Tailwind CSS を使用

Claude Code は要件を解釈し、`Write` ツールを使って `Button.tsx` を新規作成します。

1.1.4 ファイル編集

- ＞ `src/utils/helpers.ts` の `formatDate` 関数を
- ＞ ISO 8601 形式に対応するよう修正して

`Edit` ツールを使って既存ファイルを編集します。Edit ツールは**完全一致文字列置換**方式で動作します（正規表現や曖昧一致は使用しません）。ファイル全体を書き直すのではなく、変更箇所だけを差分更新します。

1.1.5 リファクタリング

- ＞ `UserList` コンポーネントのパフォーマンスを改善して。
- ＞ 不要な再レンダリングを防ぐよう `useMemo` と `useCallback` を適切に使って

1.1.6 デバッグ補助

エラーメッセージをそのまま貼り付けるだけで原因を特定します：

```
> npm run build で以下のエラーが出ます :  
> TypeError: Cannot read properties of undefined (reading 'map')  
>   at UserList (src/components/UserList.tsx:23:18)
```

1.1.7 ファイル・フォルダの参照

@ 記法でファイルやフォルダを会話に含められます (IDE 拡張のみ) :

```
@src/components/Button.tsx をレビューして  
@src/utils/ の全ファイルの依存関係を整理して
```

ターミナルから使う場合は、自然言語でファイルパスを指示するか、パイプでファイルの内容を渡します :

```
cat src/utils/helpers.ts | claude -p "この関数の問題点を教えて"
```

1.1.8 複数ファイルの同時編集

Claude Code は1回の指示で複数ファイルを同時に変更できます :

```
> Button コンポーネントの props に color プロパティを追加して。  
> Button.tsx, Button.test.tsx, Storybook の Button.stories.tsx も  
   更新して
```

1.1.9 ツール一覧

Claude Code が内部で使う主要ツール :

ツール	説明	権限要否
Read	ファイルの内容を読む	不要
Edit	ファイルの一部を完全一致置換で差分編集	要
Write	新しいファイルを作成、または既存ファイルを上書き	要
Glob	ファイルパターンでファイルを検索	不要
Grep	ファイル内容を全文検索（ripgrep ベース）	不要
Bash	シェルコマンドを実行	要
PowerShell	PowerShell コマンドを実行（Windows 環境）	要
Agent	サブエージェントを起動して独立したタスクを処理	不要
LSP	言語サーバーによるコード補完・参照解析	不要
WebFetch	URL からコンテンツを取得	要
WebSearch	Web 検索を実行	要

ツール	説明	権限要否
NotebookEdit	Jupyter ノートブックのセルを編集	要
AskUserQuestion	要件確認のための多択質問	不要
Monitor	バックグラウンドでコマンドを監視して変化に反応	要

注意: ツールの利用可能性はプランや環境によって異なります。現在セッションで使えるツールを確認するには「利用可能なツールは何ですか?」と聞いてください。

1.1.10 承認フロー

破壊的な操作は実行前に確認を求めます：

Claude: 以下のファイルを削除しようとしています：

- src/legacy/old-api.ts

実行しますか？ [y/N]

`settings.json` の `permissions.allow` に追加すると自動承認できます。

1.1.11 🧑🏻‍🔧 練習問題

1. **【確認】** Claude Code がファイルを編集する際に使う主要ツールを3つ答えてください。

2. **【実践】** 任意のプロジェクトで「このファイルのコードをリファクタリングして」と依頼してみましょう。
3. **【実践】** `> src/` のようにフォルダをコンテキストとして渡す方法を試してみましょう。
4. **【応用】** 大きなリファクタリングを依頼するとき、一度に全部頼むのとステップに分けて頼むのはどちらが良いですか？

1.1.12 次のステップ

→ 1-2: スラッシュコマンド一覧 に進む

1.2 1-2: スラッシュコマンド一覧

学習時間: 15分 | 難易度: ★★

1.2.1 スラッシュコマンドとは

スラッシュコマンドは `/` で始まるショートカットで、特定の機能やスキルを即座に呼び出せます。**組み込みコマンド**（固定のロジックを実行）と、**バンドルスキル**（Claude への指示ベースで動作）と、ユーザー定義の**カスタムスキル**の3種類があります。

`/` を入力すると補完リストが表示されます。コマンドはメッセージの先頭でのみ認識され、その後に続くテキストは引数として渡されます。

1.2.2 組み込みコマンド一覧

1.2.2.1 セッション管理

コマンド	説明
<code>/help</code>	利用可能なコマンドとヘルプを表示
<code>/clear [name]</code>	会話履歴をクリアして新しい会話を開始。以前の会話は <code>/resume</code> で復元可能。エイリアス: <code>/reset</code> , <code>/new</code>
<code>/compact [instructions]</code>	会話を要約してコンテキストを節約。引数でサマリーの焦点を指定可能
<code>/context [all]</code>	コンテキスト使用状況を色付きグリッドで可視化。最適化の提案も表示
<code>/config [key=value]</code>	設定を対話的に変更（ <code>/settings</code> でも可）。 <code>key=value</code> 形式で直接設定も可能
<code>/status</code>	バージョン・モデル・アカウント・接続状態を表示（Claude 応答中でも使用可能）
<code>/fast [on\ off]</code>	Fast モードの切り替え

コマンド	説明
<code>/model [model]</code>	AI モデルを切り替えてデフォルトとして保存
<code>/effort [level\ auto]</code>	努力レベルを調整（low / medium / high / xhigh / max / ultracode）
<code>/plan [description]</code>	プランモードに入る。引数でタスクを指定可能
<code>/exit</code>	Claude Code を終了。エイリアス: <code>/quit</code>
<code>/resume [session]</code>	過去のセッションを再開（セッション名または ID で指定）
<code>/rename [name]</code>	セッションに名前を付ける
<code>/branch [name]</code>	会話をブランチしてコピーを作成
<code>/fork <directive></code>	現在の会話を引き継いだバックグラウンドサブエージェントを起動
<code>/rewind</code>	チェックポイントに巻き戻す。エイリアス: <code>/checkpoint</code> , <code>/undo</code>
<code>/export [filename]</code>	会話をテキストファイルに書き出す

コマンド	説明
<code>/goal [condition\ clear]</code>	ゴールを設定（条件が満たされるまで Claude が自律作業を継続）
<code>/btw <question></code>	会話履歴に残さずサイドクエスチョンを送る
<code>/recap</code>	現在のセッションの要約を生成
<code>/copy [N]</code>	最後のアシスタント応答をクリップボードにコピー

1.2.2.2 ファイル・プロジェクト

コマンド	説明
<code>/init</code>	CLAUDE.md を自動生成（プロジェクト分析）
<code>/memory</code>	CLAUDE.md とオートメモリを管理
<code>/diff</code>	インタラクティブな差分ビューを開く
<code>/add-dir <path></code>	セッションに作業ディレクトリを追加
<code>/cd <path></code>	セッションの作業ディレクトリを変更

1.2.2.3 並列処理・バックグラウンド

コマンド	説明
<code>/agents</code>	サブエージェントの設定を管理
<code>/tasks</code>	バックグラウンドで実行中のタスクを表示。エイリアス: <code>/bases</code>
<code>/background [prompt]</code>	セッションをバックグラウンドに移行してターミナルを解放。エイリアス: <code>/bg</code>
<code>/batch <instruction></code>	[スキル] 大規模変更を並列サブエージェントで実行（各サブエージェントは独立した git worktree で動作）
<code>/schedule [description]</code>	ルーティン（定期実行タスク）を設定。エイリアス: <code>/routines</code>
<code>/loop [interval] [prompt]</code>	[スキル] プロンプトを定期的に繰り返し実行

1.2.2.4 コードレビュー・品質

コマンド	説明
<code>/code-review [level] [--fix] [--comment] [target]</code>	[スキル] コードレビュー（ <code>--fix</code> で自動修正、 <code>ultra</code> でクラウドレビュー）
<code>/simplify [target]</code>	[スキル] コードの簡略化・リファクタリングを適用（バグ検出は行わない）
<code>/security-review</code>	セキュリティ脆弱性を分析
<code>/review [PR]</code>	プルリクエストをローカルでレビュー
<code>/ultrareview [PR]</code>	クラウドでのディープレビュー（ <code>/code-review ultra</code> の別名）

1.2.2.5 スキル・ツール

コマンド	説明
<code>/run-skill-generator</code>	<code>/run</code> や <code>/verify</code> 用のスキルを生成
<code>/run</code>	[スキル] アプリを起動して動作確認
<code>/verify</code>	[スキル] コード変更が実際に動作することを確認
<code>/debug [description]</code>	[スキル] デバッグログを有効化して問題を診断
<code>/skills</code>	利用可能なスキルの一覧を表示

1.2.2.6 設定・診断

コマンド	説明
<code>/doctor</code>	インストールと設定を診断 (<code>f</code> キーで問題を自動修正)
<code>/permissions</code>	ツール許可ルールを管理。エイリアス: <code>/allowed-tools</code>
<code>/hooks</code>	フック設定を確認
<code>/mcp [reconnect\ enable\ disable]</code>	MCP サーバーの接続を管理
<code>/ide</code>	IDE 連携の状態を確認・管理
<code>/feedback [report]</code>	フィードバックやバグを報告。エイリアス: <code>/bug</code> , <code>/share</code>
<code>/keybindings</code>	キーボードショートカットファイルを開く
<code>/usage</code>	セッションコスト・プラン使用量・統計を表示。エイリアス: <code>/cost</code> , <code>/stats</code>

1.2.3 コマンドの使い方

1.2.3.1 引数なし

`/code-review`

カレントブランチの変更差分をレビューします。

1.2.3.2 引数あり

```
/code-review --fix  
/code-review src/components/Button.tsx
```

1.2.3.3 スキルとして呼び出す

独自スキルも同じようにスラッシュで呼び出せます：

```
/my-custom-skill
```

1.2.4 /fast モードについて

/fast コマンドは Fast モードのオン/オフを切り替えます。Fast モードは特定のプランや環境での高速応答モードです。

```
/fast on    # Fast モードを有効化  
/fast off   # Fast モードを無効化  
/fast       # 現在の状態をトグル
```

1.2.5 /plan の使い方

/plan コマンドはプランモードに入り、コードを変更する前にアプローチを設計できます：

```
/plan 認証バグを修正する
```

Claude はコードに触れずに計画を立て、承認後に実装を開始します。

1.2.6 /init の使い方

プロジェクト初回セットアップ時に使います：

```
claude  
/init
```

Claude Code がプロジェクトを分析して `CLAUDE.md` を生成します。生成される内容： - プロジェクト概要 - 技術スタック - よく使うコマンド - コーディング規約の推測

1.2.7 /config の使い方

```
/config
```

対話的に設定できる項目： - モデルの選択 - テーマ（light / dark） - 自動コンパクト設定 - 通知設定

`key=value` 形式での直接設定も可能：

```
/config theme=dark  
/config model=sonnet  
/config thinking=false
```

1.2.8 カスタムスラッシュコマンド

`.claude/skills/<name>/SKILL.md` を配置すると `/name` で呼び出せます：

```
.claude/skills/  
└─ deploy-check/  
    └─ SKILL.md    → /deploy-check として使える
```

`.claude/commands/deploy.md` のような旧形式のカスタムコマンドも引き続き動作します。スキルはカスタムコマンドと同名の場合、スキルが優先されます。

1.2.9 /help の出力例

実際のコマンド一覧はバージョンや環境によって異なります。`/` を入力すると補完リストが表示されます。

```
/help          Show help and available commands  
/clear         Start a new conversation  
/compact       Free up context by summarizing  
/config        Open the Settings interface  
/init          Initialize project with a CLAUDE.md guide  
/model         Switch the AI model  
/effort        Set the model effort level  
/code-review   Review the current diff  
/run           Launch and drive your project's app  
/verify        Confirm a code change works  
/debug         Enable debug logging  
/doctor        Diagnose your Claude Code installation  
/memory        Edit CLAUDE.md memory files  
/agents        Manage agent configurations  
...
```

Custom skills:

```
/deploy-check (from .claude/skills/deploy-check/)
```

1.2.10 🏆 練習問題

1. 【確認】 `/clear` と `/reset` の違いを説明してください。
2. 【実践】 `/help` を実行して利用可能なコマンド一覧を確認しましょう。
3. 【実践】 `/code-review` を実行してみましょう。どのような出力が返りますか？
4. 【応用】 長い会話セッションで `/compact` を使うのはどのような状況ですか？

1.2.11 次のステップ

→ 1-3: ターミナル・Bash ツール に進む

1.3 1-3: ターミナル・Bash ツール

学習時間: 15分 | 難易度: ★★

1.3.1 Bash ツールとは

Claude Code は `Bash` ツールを通じてシェルコマンドを実行できます。これにより、ビルド・テスト・デプロイなどの実際の作業を自律的に行えます。

1.3.2 Windows での PowerShell ツール

Windows 環境では、`PowerShell` ツールが追加で利用できます。

環境	動作
Git Bash なし (Windows)	PowerShell ツールが自動で有効化
Git Bash あり (Windows)	段階的ロールアウト中
Linux / macOS / WSL	<code>CLAUDE_CODE_USE_POWERSHELL_TOOL=1</code> で有効化可能

`settings.json` で明示的に有効化することもできます：

```
{
  "env": {
    "CLAUDE_CODE_USE_POWERSHELL_TOOL": "1"
  }
}
```

PowerShell ツールが有効な場合、Claude は PowerShell をプライマリシェルとして扱います。Git Bash がインストールされていれば、POSIX スクリプト用に Bash ツールも引き続き使用できます。

1.3.3 基本的な使い方

```
> npm のテストを実行して、失敗しているテストを教えてください
```

Claude Code は内部で以下を実行します：

```
npm test
```

出力を解析して失敗しているテストを特定し、修正案を提示します。

1.3.4 Git 操作

＞ 変更内容を確認してコミットメッセージを提案して

```
# Claude Code が実行する一連のコマンド
git status
git diff
git diff --staged
# → コミットメッセージを提案
```

＞ 現在のブランチの変更を main にマージして、コンフリクトがあれば解決して

1.3.5 ビルドとテスト

＞ ビルドして、エラーがあれば修正して

```
npm run build
# エラーを解析 → 関連ファイルを修正 → 再ビルド
```

1.3.6 パッケージ管理

＞ date-fns をインストールして、既存の日付処理を date-fns に置き換えて

```
npm install date-fns
# → 関連ファイルを自動編集
```

1.3.7 権限とセキュリティ

1.3.7.1 自動承認のルール

デフォルトでは危険な操作は確認が必要です。`settings.json` の `permissions` セクションで細かく制御できます：

```
{
  "permissions": {
    "allow": [
      "Bash(git *)",
      "Bash(npm run *)",
      "Bash(npm test)",
      "Read(**)",
      "Edit(**)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)"
    ]
  }
}
```

1.3.7.2 パターン構文

パターン	意味
<code>Bash(git *)</code>	git で始まる全コマンドを許可
<code>Bash(npm run *)</code>	npm run から始まる全コマンドを許可
<code>Read(**)</code>	全ファイルの読み込みを許可
<code>Edit(/src/**)</code>	src/ 以下のファイル編集を許可
<code>Bash(rm -rf *)</code>	rm -rf を拒否
<code>WebFetch(domain:example.com)</code>	特定ドメインへの WebFetch を許可

`Edit(...)` の許可ルールは同じパスへの Read アクセスも自動的に許可します。

1.3.7.3 CLI フラグでの設定

セッション単位での制御は CLI フラグでも可能です：

```
# 特定コマンドを確認なしで実行
claude --allowedTools "Bash(git log *)" "Bash(git diff *)"

# 特定ツールを拒否
claude --disallowedTools "Bash(rm *)" "Edit"
```

1.3.7.4 危険なコマンドを防ぐ方法

`deny` ルールに追加することで、特定の危険なコマンドを常に拒否できます：

```
{
  "permissions": {
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(DROP TABLE *)"
    ]
  }
}
```

また、`--permission-mode` フラグで動作モードを制御できます： - `default` — デフォルト（要確認） - `acceptEdits` — ファイル編集を自動承認 - `plan` — 計画のみ（実行しない） - `bypassPermissions` — すべて自動承認（危険・開発環境のみ推奨）

1.3.8 バックグラウンド実行

長時間かかるコマンドはバックグラウンドで実行できます：

＞ 開発サーバーをバックグラウンドで起動して、起動完了を確認して

```
npm run dev &
# 起動ログを監視 → "Server running" を検出 → 確認完了
```

バックグラウンドタスクの管理は `/tasks` コマンドで行います。

1.3.9 複数コマンドの連携

＞ 依存関係をインストールして、ビルドして、テストを実行して、全部成功したら `git commit` して

```
npm install && npm run build && npm test && git add -A && git commit -m "..."
```

Claude Code は各ステップの結果を確認し、失敗した場合は対処してから次に進みます。

1.3.10 Bash ツールの制限

各コマンドには以下の制限があります：

- **タイムアウト**: デフォルト2分（最大10分まで拡張可能）
- **出力長**: デフォルト 30,000 文字（環境変数 `BASH_MAX_OUTPUT_LENGTH` で調整可能）

1.3.11 よく使うパターン

1.3.11.1 プロジェクトの現状把握

```
> このプロジェクトのテストカバレッジを計測して
> npm run test:coverage
```

1.3.11.2 リント・フォーマット

```
> ESLint と Prettier を実行して、自動修正できるものは直して
> npm run lint:fix && npm run format
```

1.3.11.3 ポート確認

```
> 3000番ポートを使っているプロセスを教えて
```

1.3.12 🏆 練習問題

1. **【確認】** Bash ツールと PowerShell ツールの違いを説明してください（Windows 環境の場合）。
2. **【実践】** `git status` を Claude Code 経由で実行してみましょう。
3. **【実践】** `settings.json` に `allowedTools` を設定して、特定コマンドを確認なしで実行できるようにしてみましょう。
4. **【応用】** 危険なコマンド（`rm -rf` 等）の実行を防ぐにはどうすれば良いですか？

1.3.13 次のステップ

→ 1-4: コンテキスト管理 に進む

1.4 1-4: コンテキスト管理

学習時間: 15分 | 難易度: ★★

1.4.1 コンテキストとは

Claude Code の会話ウィンドウには上限（コンテキストウィンドウ）があります。現在のコンテキストウィンドウは **200,000 トークン**です。長い作業では会話が圧縮・要約されることがあります。効果的なコンテキスト管理が生産性を左右します。

1.4.1.1 セッション開始時に自動読み込まれるもの

セッション開始時には以下が自動的にコンテキストに読み込まれます：

内容	説明
システムプロンプト	Claude Code の基本動作・ツール使用の指示（約 4,200 トークン）
Auto Memory (MEMORY.md)	過去のセッションから蓄積された Claude 自身のノート（最大 200 行 / 25KB）
環境情報	作業ディレクトリ・プラットフォーム・シェル・OS バージョン
CLAUDE.md	プロジェクト設定ファイル（存在する場合）

1.4.2 コンテキストの確認

```
/context
```

現在のコンテキスト使用状況を色付きグリッドで可視化します。 `/context all` で詳細な内訳を表示します。

```
/status
```

現在のトークン使用量と残量が表示されます。

1.4.3 コンテキストのリセット

```
/clear
```

会話履歴をクリアして新しいセッションを開始します。CLAUDE.md・メモリファイル・Auto Memory の内容は保持されます。

1.4.4 手動コンパクト

```
/compact
```

会話をその場でサマリーに圧縮し、コンテキストを節約します。焦点を指定することも可能です：

```
/compact 認証フローの実装に集中して要約して
```

1.4.4.1 コンパクト後も保持されるもの

`/compact` 実行後も以下の情報は保持されます： - CLAUDE.md・メモリファイルの内容 - セッション内で適用されたルールとスキル - 重要な決定事項やコード変更の要約

1.4.5 自動コンパクト

コンテキストが一定量（コンテキストウィンドウの上限付近）に達すると、Claude Code は自動的に過去の会話を要約して圧縮します。この自動コンパクト機能は `/config` でカスタマイズできます。

1.4.6 プランモード

大きな変更を加える前に、コンテキストを節約しながら計画を立てることができます：

```
/plan 認証システムを OAuth 2.0 に移行する
```

プランモードでは Claude はコードを変更せずに計画だけを立て、承認後に実装を開始します。これにより、方向性の確認をコンテキストを大量消費する前に行えます。

1.4.7 CLAUDE.md による常時コンテキスト

`CLAUDE.md` はセッション開始時に常に読み込まれます。ここに書かれた内容は `/clear` 後も有効です：

```
# My Project

## 技術スタック
- React 18 + TypeScript
- Tailwind CSS
- Vitest

## よく使うコマンド
- `npm run dev` - 開発サーバー（ポート 3000）
- `npm test` - テスト実行

## コーディング規約
- コンポーネントは関数コンポーネントのみ
- Props 型は interface で定義
- テストは Vitest + Testing Library
```

1.4.8 ファイルの明示的な読み込み

大きなプロジェクトでは、関連ファイルだけを指示して読み込ませると効率的です：

> `src/auth/` ディレクトリだけ読んで、認証フローを説明して

> `package.json` と `tsconfig.json` を読んで、プロジェクトの設定を確認して

複数のファイルを一度に渡すことも可能です：

> `src/components/Button.tsx` と `src/types/index.ts` を読んで、型の整合性を確認して

1.4.9 サブエージェントによるコンテキスト保護

長い調査タスクは Agent（サブエージェント）に委ねることでメインのコンテキストを消費せずに済みます。サブエージェントは独立したコンテキストウィンドウで動作し、結果だけをメイン会話に返します：

> `src/components/` 以下の全コンポーネントを調査して、
> `props` の型定義に不整合がないか確認するエージェントを起動して

1.4.10 セッションをまたいだ継続

セッションが終わっても作業を継続するには：

1. **CLAUDE.md** に進行状況を書いておく
2. **メモリシステム** に重要な決定を保存する
3. `/resume` で過去のセッションを再開する

1.4.11 効果的なコンテキスト管理のコツ

方法	効果
CLAUDE.md に技術スタックを書く	毎回説明不要
<code>/clear</code> を適切に使う	コンテキスト汚染を防ぐ
大きなタスクを分割する	1セッションの集中度を高める
<code>/compact</code> でサマリー化する	コンテキストを節約しながら継続
サブエージェントに調査を委ねる	メインコンテキストを保護
不要なファイルを読ませない	トークンを節約
<code>/context</code> で使用量を監視する	残量を把握して計画的に作業

1.4.12 🏆 練習問題

1. **【確認】** Claude Code がコンテキストを圧縮するのはどのような状況ですか？
2. **【実践】** 複数のファイルを一度にコンテキストに渡す方法を試してみましょう。
3. **【実践】** `/compact` を実行して、会話の圧縮がどのように行われるか観察しましょう。
4. **【応用】** 大きなりポジトリで作業する場合、コンテキストを効果的に管理するための戦略を考えてみましょう。

1.4.13 次のステップ

→ 2-1: スキルとは に進む

2. スキルシステム

2.1 2-1: スキルとは

学習時間: 15分 | 難易度: ★★

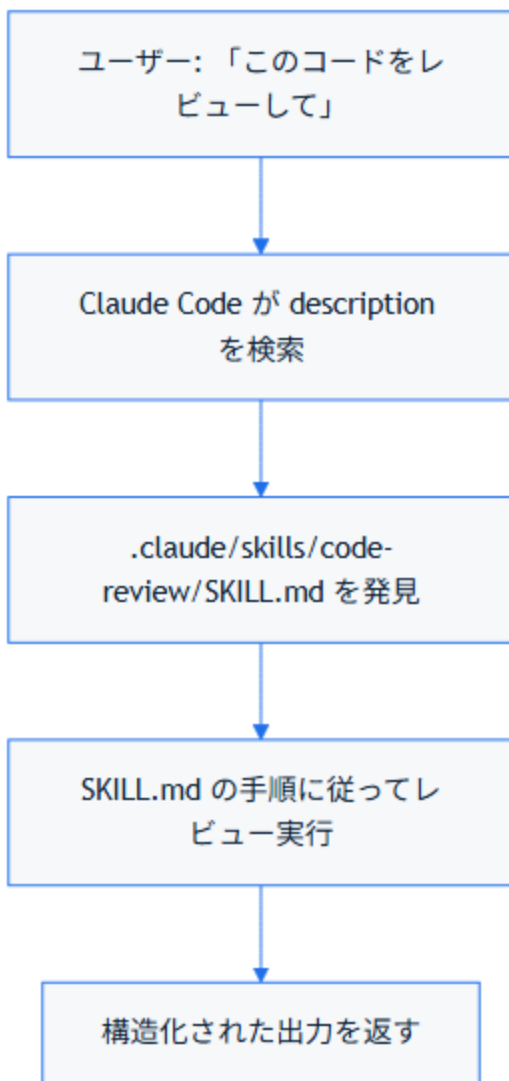
2.1.1 概要

スキル (Skills) は、Claude Code に特定のタスクを実行させるための指示書です。SKILL.md というファイルに手順・ルール・出力形式を記述し、Claude Code が自動的に読み込んで実行します。

スキルは Agent Skills オープンスタンダード (agentskills.io) に基づく共通フォーマットで、Claude Code と他の AI ツールの両方で使えます。

ポイント: カスタムコマンド (.claude/commands/ 以下のファイル) はスキルに統合されました。既存の .claude/commands/ ファイルはそのまま動作し、スキルと同じ /コマンド名 で呼び出せます。

2.1.2 スキルの仕組み



2.1.3 SKILL.md の構造

```
---
name: code-review
description: コードの品質・セキュリティ・パフォーマンスをレビューしま
す
tags:
  - review
  - quality
---

# コードレビュースキル

## 概要
指定されたコードを複数の観点でレビューし、改善点を提示します。

## 手順
1. コードを読み込む
2. 品質・セキュリティ・パフォーマンスの観点でチェック
3. 優先度付きで改善点を列挙

## 出力形式
...
```

2.1.4 フロントマターの役割

フロントマターの全フィールドはオプションです。ただし `description` は強く推奨されます。

フィールド	必須	役割
<code>name</code>	任意	スキル一覧に表示される名前。デフォルトはディレクトリ名
<code>description</code>	推奨	自動読み込みの判断に使う説明文。具体的に書く
<code>tags</code>	任意	カテゴリ分類（検索・発見に使用）
<code>disable-model-invocation</code>	任意	<code>true</code> にすると Claude が自動起動しなくなる
<code>allowed-tools</code>	任意	スキル実行中に許可するツール
<code>context</code>	任意	<code>fork</code> を指定するとサブエージェントで実行

注意: `id` フィールドは Agent Skills 標準の拡張で、Claude Code の公式フロントマター仕様には含まれません。`name` フィールドが表示名として機能し、コマンド名はディレクトリ名から決まります。

`description` が特に重要です。Claude Code はこのフィールドを見て「今の作業にこのスキルが必要か」を判断します。

2.1.5 スキルの配置場所

種類	パス	適用範囲
個人用	<code>~/.claude/skills/<name>/SKILL.md</code>	全プロジェクト
プロジェクト用	<code>.claude/skills/<name>/SKILL.md</code>	そのプロジェクトのみ
エンタープライズ	管理設定参照	組織内全ユーザー

補足: ネストされたディレクトリにも配置できます。
`packages/frontend/.claude/skills/` に置けば、そのパッケージで作業するときだけ利用可能になります（モノレポ向け）。

2.1.6 スキルが起動するタイミング

1. 自動読み込み

会話の内容がスキルの `description` にマッチすると自動的に読み込まれます。

2. スラッシュコマンドで明示呼び出し

```
/code-review
/debug
/my-custom-skill
```

3. 自動起動を無効にする

デプロイなど副作用のある操作は `disable-model-invocation: true` を設定してユーザーが手動で起動することを推奨します。

2.1.7 ディレクトリ構造

```
.claude/skills/
├─ code-review/
│   ├── SKILL.md           # メインの指示（必須）
│   ├── scripts/          # 補助スクリプト（任意）
│   │   └─ check.sh
│   ├── references/        # 参照ドキュメント（任意）
│   │   └─ style-guide.md
│   └─ assets/             # テンプレート等（任意）
│       └─ report.md
```

2.1.8 組み込みスキル vs カスタムスキル

種類	説明	例	適用場面
バンドルスキル	Claude Code に標準搭載	<code>/code-review</code> , <code>/debug</code> , <code>/run</code>	設定不要ですぐ使える
プロジェクトスキル	<code>.claude/skills/</code> に自分で追加	<code>/deploy-check</code>	チームで共有したいワークフロー
個人スキル	<code>~/.claude/skills/</code> に自分で追加	<code>/my-workflow</code>	自分だけのカスタム手順

2.1.8.1 バンドルスキル

Claude Code にあらかじめ搭載されているスキルです。インストール直後から `/code-review` や `/run` などがすぐに使えます。設定・追加作業は一切不要です。 `disableBundledSkills` 設定で無効化できます。

2.1.8.2 プロジェクトスキル

リポジトリ内の `.claude/skills/` に配置するスキルです。デプロイ手順やチーム固有のレビュー基準など、**プロジェクト固有のワークフロー**を定義するのに適しています。git で管理することでチームメンバー全員が同じスキルを共有できます。

2.1.8.3 個人スキル

`~/ .claude/skills/` に自分で追加するスキルです。よく使う個人的な作業手順やフォーマットなど、**自分だけのカスタムワークフロー**を定義します。すべてのプロジェクトに横断して適用されます。

2.1.9 Agent Skills オープンスタンダードとの互換性

スキルは agentskills.io のオープンスタンダードに準拠しており、複数の AI ツールで互換性があります。Claude Code はさらにサブエージェント実行・動的コンテキスト注入・起動制御などの拡張機能を追加しています。

2.1.10 🛠️ 練習問題

1. **【確認】** `description` フィールドが特に重要とされる理由を説明してください。
2. **【確認】** プロジェクトスキルと個人スキルの保存場所の違いを説明してください。
3. **【実践】** `~/ .claude/skills/` ディレクトリを確認し、インストール済みのスキルを一覧表示しましょう。

4. 【応用】 チームで統一したコーディング規約チェックを自動化するには、スキルをどこに配置するのが適切ですか？

2.1.11 次のステップ

→ 2-2: /skill-creator ハンズオン に進む

2.2 2-2: /skill-creator ハンズオン

学習時間: 20分 | 難易度: ★★

2.2.1 skill-creator とは

`skill-creator` は、スキルのテスト・評価・改善サイクルを自動化する **Claude Code 公式プラグイン** です。対話形式でスキルの要件を引き出し、テストケースを作成・実行して SKILL.md を評価・改善します。

注意: `skill-creator` は Claude Code に標準搭載されているバンドルスキルではなく、公式プラグインマーケットプレイスからインストールするプラグインです。

2.2.2 インストール方法

```
/plugin install skill-creator@claude-plugins-official
```

インストール後、現在のセッションで使えるようにします：

```
/reload-plugins
```

プラグインが見つからない場合はマーケットプレイスを更新してください：

```
/plugin marketplace update claude-plugins-official
```

または初めて追加する場合：

```
/plugin marketplace add anthropics/claude-plugins-official
```

2.2.3 使い方

インストール後、Claude にスキルの評価を依頼します：

```
> my-skill スキルを skill-creator で評価して  
> evaluate my summarize-changes skill with skill-creator
```

2.2.4 評価の仕組み

`skill-creator` は以下のプロセスでスキルを自動評価・改善します：

ステップ	内容
テストケース作成	プロンプト・入力ファイル・期待動作を <code>evals/evals.json</code> に保存
独立実行	テストケースごとにサブエージェントを生成してクリーンな状態で実行
採点	各アサーションを出力と照合し <code>grading.json</code> に合否を記録
ベンチマーク	スキルあり/なしの通過率・時間・トークン数を <code>benchmark.json</code> に集計
バージョン比較	2バージョンのスキルをブラインド A/B テスト
description 調整	起動すべきプロンプト/すべきでないプロンプトを生成し、ヒット率を測定
レビュービューア	出力を検査・フィードバックを記録できる HTML レポートを生成

2.2.5 生成されるファイル構造

```
.claude/skills/my-skill/
├─ SKILL.md          # メイン指示書
└─ evals/
    ├─ evals.json     # テストケース定義
    ├─ grading.json   # 各アサーションの合否
    └─ benchmark.json # スキルあり/なしのベンチマーク結果
```

2.2.6 テストケースの形式（evals.json）

`skill-creator` が管理するテストケースは以下の形式です：

```
{
  "test_cases": [
    {
      "prompt": "以下の Python コードをセキュリティチェックしてください：\n...",
      "assertions": [
        "SQL インジェクションの脆弱性を高重大度で検出すること",
        "修正例（パラメータ化クエリ）を提示すること",
        "Markdown 表形式で出力されること"
      ]
    }
  ]
}
```

2.2.7 手動でスキルを作成する場合

`skill-creator` を使わずに直接 SKILL.md を書く方法は 2-4: カスタムスキル作成 を参照してください。

2.2.8 ベースラインとの比較

`skill-creator` の評価では、同じプロンプトをスキルあり/なしで実行して結果を比較します。スキルの効果を定量的に確認できます：

- **通過率の改善:** スキルあり vs なしで何%改善されたか
- **トークンコスト:** スキルの追加によるコスト増加
- **実行時間:** スキルによる処理時間への影響

2.2.9 Tips

- **description は長めに:** 自動トリガーの精度が上がる
- **テストケースは現実的に:** 実際に起きうる入力を使う
- **フレッシュセッションで確認:** 作成時のコンテキストがマスクする問題を見逃さないよう、新しいセッションでテストする

2.2.10 🏆 練習問題

1. **【確認】** `skill-creator` の主な用途は「新規スキルの作成」と「既存スキルの評価・改善」のどちらですか？その理由を本文から説明してください。
2. **【確認】** `skill-creator` がスキルを評価する際に生成する主なファイルを2つ答えてください。
3. **【実践】** `skill-creator` プラグインをインストールし、手持ちのスキルに対して評価を実行してみましょう。
4. **【応用】** 自分が作成したカスタムスキルの品質を継続的に改善するために、`skill-creator` をどのように活用しますか？

2.2.11 次のステップ

→ 2-3: 組み込みスキル活用 に進む

2.3 2-3: 組み込みスキル活用

学習時間: 20分 | 難易度: ★★

2.3.1 バンドルスキルとは

バンドルスキルは Claude Code に標準搭載されており、インストール直後からすぐに使えるスキルです。組み込みコマンド（`/help`、`/compact` 等）とは異なり、バンドルスキルはプロンプトベースで動作します。Claude が詳細な指示を受け取り、ツールを使って作業を調整します。

`disableBundledSkills` 設定で全バンドルスキルを無効化できます。

2.3.2 バンドルスキル一覧

スキル	コマンド	用途
code-review	<code>/code-review</code>	コードレビュー（バグ・クリーンアップ検出）
simplify	<code>/simplify</code>	コード整理・簡略化（バグ検出なし）
security-review	<code>/security-review</code>	セキュリティレビュー
debug	<code>/debug</code>	デバッグログの有効化とセッション診断
run	<code>/run</code>	アプリ起動・動作確認
verify	<code>/verify</code>	コード変更の動作確認
run-skill-generator	<code>/run-skill-generator</code>	<code>/run</code> ・ <code>/verify</code> 用レシピ生成
batch	<code>/batch</code>	コードベース全体への並列大規模変更
loop	<code>/loop</code>	繰り返し実行
claude-api	<code>/claude-api</code>	Claude API リファレンス読み込み

スキル	コマンド	用途
fewer-permission-prompts	<code>/fewer-permission-prompts</code>	許可プロンプトの削減

2.3.3 /code-review

現在のブランチ差分またはファイルをレビューします。

```
# ブランチ全体をレビュー
/code-review

# 特定ファイルをレビュー
/code-review src/components/Button.tsx

# レビューして自動修正
/code-review --fix

# PR にインラインコメントを投稿
/code-review --comment

# クラウドでマルチエージェントレビュー
/code-review ultra
```

レビュー観点： - 正確性（バグ・ロジックエラー） - 再利用性・簡潔さ - パフォーマンス - セキュリティ

努力レベル指定：

```
/code-review low      # 高確信度の指摘のみ
/code-review high     # 広範なカバレッジ
/code-review ultra     # クラウドでマルチエージェント深層レビュー
```

2.3.4 /simplify

変更されたコードの重複・複雑さ・非効率を検出し、自動改善します（バグ検出は行いません）。

```
/simplify

# 特定ファイルまたは PR を対象に
/simplify src/utils.ts
```

4つのサブエージェントが並列で実行します： - 既存ヘルパーの再利用 - コードの単純化 - 効率化 - 抽象レベルの適正化

注意: バグを探すには `/code-review` を使ってください。`/simplify` は品質向上のみです。

2.3.5 /debug

セッションのデバッグログを有効化して問題を診断します。

```
# デバッグログを有効化
```

```
/debug
```

```
# 問題の説明を渡してフォーカス分析
```

```
/debug TypeError: Cannot read properties of undefined at UserLis  
t.tsx:23
```

重要: `/debug` は Claude Code 自体のデバッグ診断ツールです。コードのバグ修正を依頼する場合は、直接 Claude に問題を説明してください。

2.3.6 /run

アプリを起動して実際に動作を確認します。

```
/run
```

プロジェクトの種類（React, Node.js, Python 等）を自動検出し、適切な起動コマンドを実行します。フロントエンドの変更はブラウザで確認し、スクリーンショットを取得します。

複雑な起動手順（データベース・環境ファイル・マルチステップビルド）がある場合は `/run-skill-generator` で事前にレシピを記録しておくと確実です。

2.3.7 /verify

コードの変更が期待通りに動作するか、実際にアプリを起動して確認します。

```
/verify
```

テストやタイプチェックだけでなく、実際の動作で確認します。

2.3.8 /security-review

現在のブランチ差分のセキュリティ上の問題を検出します。

```
/security-review
```

チェック項目： - SQL インジェクション - XSS（クロスサイトスクリプティング） - CSRF - 認証・認可の問題 - 機密情報のハードコード - 依存パッケージの脆弱性

2.3.9 /code-review ultra（クラウドマルチエージェントレビュー）

複数のサブエージェントがクラウドで並列レビューを行う高精度モードです。`/ultrareview` は現在 `/code-review ultra` のエイリアスになっています：

```
/code-review ultra

# GitHub PR 番号を指定
/code-review ultra 42
```

通常の `/code-review` より深い分析が行われますが、時間とトークンを消費します。Pro・Max プランで 3 回まで無料です。

2.3.10 スキルのカスタマイズ

バンドルスキルは上書きできます。`.claude/skills/code-review/SKILL.md` を作成すると、プロジェクト固有の動作に変更できます：

プロジェクト専用 コードレビュー

追加チェック項目

- Tailwind クラスの順序 (Prettier プラグインに準拠)
- コンポーネントの最大行数 (100行以下)
- テストファイルの同時更新確認

(以下、標準のレビュー観点...)

2.3.11 🏆 練習問題

1. **【確認】** 組み込みスキルとカスタムスキルの違いを説明してください。
2. **【実践】** `/code-review` を実行して、出力の構造を確認しましょう。
3. **【実践】** `/security-review` を任意のファイルに対して実行してみましょう。
4. **【応用】** `/code-review` と `/security-review` を使い分ける基準を考えてみましょう。

2.3.12 次のステップ

→ 2-4: カスタムスキル作成 に進む

2.4 2-4: カスタムスキル作成

学習時間: 25分 | 難易度: ★★

2.4.1 手書き SKILL.md の基本

`/skill-creator` プラグインを使わず、テキストエディタで直接 SKILL.md を作成することもできます。シンプルなスキルや、細かい制御が必要な場合に適しています。

2.4.2 SKILL.md テンプレート

```
---
name: deploy-check
description: デプロイ前に環境変数・ビルド・テストをチェックします。deploy, production, リリースという言葉で起動します。
tags:
  - deploy
  - ci
  - quality
---

# デプロイ前チェックスキル

## 概要
本番デプロイ前に必要なチェックを自動実行します。

## チェック項目
1. 必須環境変数の確認
2. ビルドの成功確認
3. テストの全パス確認
4. セキュリティスキャン

## 手順

### Step 1: 環境変数チェック
以下の環境変数が設定されているか確認する：
- `DATABASE_URL`
- `API_KEY`
- `NODE_ENV=production`

### Step 2: ビルド実行
```bash
```

```
npm run build
```

```
```
```

エラーがあれば報告し、修正を提案する。

Step 3: テスト実行

```
```bash
```

```
npm test
```

```
```
```

失敗したテストがあれば一覧を表示する。

Step 4: 結果サマリー

以下の形式で結果を出力する：

| チェック項目 | 結果 | 詳細 |
|--------|-------|-------|
| ----- | ----- | ----- |
| 環境変数 | 🟢/🔴 | ... |
| ビルド | 🟢/🔴 | ... |
| テスト | 🟢/🔴 | ... |

出力形式

Markdown の表形式でチェック結果を表示する。全項目🟢の場合のみ「デプロイ可」と表示する。

2.4.3 配置とテスト

```
mkdir -p .claude/skills/deploy-check/
```

```
# SKILL.md を上記の内容で作成
```

```
# テスト実行
```

```
claude
```

```
/deploy-check
```

2.4.4 フロントマターの全フィールド

フロントマターの全フィールドはオプションです。よく使うフィールドは以下の通りです：

| フィールド | 説明 |
|---------------------------------------|--------------------------------------|
| <code>name</code> | 一覧表示名（コマンド名はディレクトリ名から決まる） |
| <code>description</code> | 自動起動の判断に使う説明（推奨） |
| <code>disable-model-invocation</code> | <code>true</code> でユーザーのみ起動可に |
| <code>allowed-tools</code> | 承認なしで使えるツール |
| <code>context</code> | <code>fork</code> でサブエージェント実行 |
| <code>argument-hint</code> | 補完時のヒント（例： <code>[filename]</code> ） |
| <code>paths</code> | スキルが有効なファイルパターン（glob） |

2.4.5 ベストプラクティス

2.4.5.1 description を充実させる

自動トリガーの精度は `description` フィールドに直結します：

悪い例

description: デプロイチェックをします

良い例

description: >

本番デプロイ前の事前チェックを実行します。

環境変数・ビルド・テスト・セキュリティを確認します。

"deploy", "デプロイ", "production", "リリース", "本番" という言葉が出たときに自動的に提案します。

`description` と `when_to_use` フィールドの合計は 1,536 文字で切り捨てられます。重要なキーワードを先頭に置いてください。

2.4.5.2 手動起動専用にする

デプロイ・コミットなど副作用のある操作は自動起動させないようにします：

2.4.5.3 引数を受け取る

GitHub Issue `$ARGUMENTS` をコーディング規約に従って修正する。

1. Issue の内容を読む
2. 要件を把握する
3. 修正を実装する
4. テストを書く
5. コミットを作成する

`/fix-issue 123` と入力すると `$ARGUMENTS` が `123` に置換されます。

2.4.5.4 明確な出力形式を定義する

曖昧な指示より、具体的な出力形式を定義するほうが一貫した結果が得られます：

出力形式

必ず以下の JSON 形式で出力すること：

```
``json
{
  "status": "ok" | "warning" | "error",
  "checks": [
    {
      "name": "チェック名",
      "result": "pass" | "fail" | "warn",
      "message": "詳細メッセージ"
    }
  ],
  "summary": "全体のサマリー"
}
``
```

2.4.5.5 失敗ケースを明示する

エラーハンドリング

- ファイルが見つからない場合：エラーを報告してスキップ
- コマンドが失敗した場合：エラーログ全文を表示
- タイムアウト（30秒以上）の場合：中断してユーザーに通知

2.4.5.6 SKILL.md を簡潔に保つ

SKILL.md は 500 行以内を目安にします。詳細なリファレンスは別ファイルに分離して参照します：

```
.claude/skills/deploy-check/  
├─ SKILL.md  
├─ references/  
│   └─ deployment-checklist.md  
└─ scripts/  
    ├── check-env.sh  
    └─ run-security-scan.sh
```

SKILL.md 内で参照する：

```
## 環境変数チェック  
詳細は [references/deployment-checklist.md](references/deployment-checklist.md) を参照。  
`scripts/check-env.sh` を実行して環境変数の状態を確認する。
```

2.4.5.7 動的コンテキスト注入

！コマンド 構文でシェルコマンドを実行し、出力を SKILL.md に埋め込みます：

```
## 現在の変更  
!`git diff HEAD`  
  
## 指示  
上記の差分を2〜3点で要約し、リスク（エラー処理漏れ・ハードコード値等）を列挙する。
```

2.4.6 スキルのテスト方法

1. **直接呼び出し**: `/deploy-check` で手動テスト
2. **自動起動テスト**: スキルの `description` に合う言葉で会話し、自動起動するか確認
3. **フレッシュセッションで確認**: 新しいセッションでテストして作成時のコンテキストの影響を排除
4. **skill-creator プラグイン**: 定量的な評価が必要な場合はプラグインを使う (2-2 参照)

2.4.7 スキルの共有

2.4.7.1 チームで共有する方法

1. `.claude/skills/` をリポジトリに含める (`.gitignore` に追加しない)
2. `README.md` にスキルの説明を記載する

2.4.7.2 個人スキルとして保存する

全プロジェクトで使いたいスキルは個人スキルとして保存：

```
mkdir -p ~/.claude/skills/my-workflow/  
# SKILL.md を作成
```

2.4.8 🏆 練習問題

1. **【確認】** SKILL.md の `description` フィールドが重要な理由を説明してください。
2. **【実践】** 自分のよく使う作業（例：PR 説明文の生成）をカスタムスキルとして作成してみましょう。

3. **【実践】** 作成したスキルを `.claude/skills/` に配置し、スラッシュコマンドで呼び出しましょう。
4. **【応用】** スキルの `description` にトリガーキーワードを含めることで、どのようなメリットがありますか？

2.4.9 次のステップ

→ 3-1: フックの概要 に進む

3. フックと自動化

3.1 3-1: フックの概要

学習時間: 15分 | 難易度: ★★

3.1.1 フック（Hooks）とは

フックは、Claude Code の特定のイベントに対して自動的にシェルコマンドを実行する仕組みです。Claude Code が行動するたびに、決められた処理を自動で挟み込みます。

3.1.2 フックで実現できること

| 用途 | 例 |
|----------|-------------------------|
| 自動フォーマット | ファイル編集後に Prettier を実行 |
| 自動テスト | コード変更後に npm test を実行 |
| 通知 | Claude が完了したら Slack に通知 |
| ログ記録 | 全ツール呼び出しをファイルに記録 |
| セキュリティ | 危険なコマンドをブロック |
| 品質ゲート | lint が通らなければ編集を拒否 |

3.1.3 フックのイベント種類

| イベント | 発火タイミング |
|------------------|---------------------------|
| PreToolUse | ツール実行の直前（ブロック可能） |
| PostToolUse | ツール実行の直後 |
| Stop | Claude Code がレスポンスを完了したとき |
| SubagentStop | サブエージェントが完了したとき |
| Notification | 通知が発生したとき |
| UserPromptSubmit | ユーザーがメッセージを送信したとき（ブロック可能） |
| PreCompact | コンテキスト圧縮の直前（ブロック可能） |

3.1.4 基本的な設定方法

フックは `settings.json` に記述します：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "npm run format"
          }
        ]
      }
    ]
  }
}
```

この例では、`Edit` ツール（ファイル編集）の直後に `npm run format` が自動実行されます。

3.1.5 フックの実行結果

フックのコマンドは、exit code で成功/失敗を伝えます：

| exit code | 意味 |
|-----------|--|
| 0 | 成功。stdout の JSON を解析して処理続行 |
| 2 | ブロックエラー。イベントに応じてアクションをブロック（PreToolUse ではツール実行をブロック、UserPromptSubmit ではプロンプトをブロック等） |
| その他 | 非ブロックエラー。stderr の先頭行をトランスクリプトに表示するが処理は続行 |

3.1.6 フックの出力

フックのコマンドが標準出力に JSON を出力すると、Claude Code はその内容を解析して動作を制御できます。主な出力フィールド：

```
{
  "continue": true,
  "stopReason": "終了理由 (continue: false の場合) ",
  "suppressOutput": false,
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "allow|deny|ask|defer",
    "permissionDecisionReason": "理由の説明",
    "additionalContext": "Claude に渡す追加コンテキスト"
  }
}
```

たとえば PreToolUse でツールをブロックするには：

```
#!/bin/bash
jq -n '{
  hookSpecificOutput: {
    hookEventName: "PreToolUse",
    permissionDecision: "deny",
    permissionDecisionReason: "このコマンドは禁止されています"
  }
}'
```

Claude Code はこの JSON を受け取り、次のアクションに反映します。

3.1.7 設定ファイルの場所

| ファイル | 適用範囲 |
|--|-----------------|
| <code>~/.claude/settings.json</code> | 全プロジェクト共通 |
| <code>.claude/settings.json</code> | そのプロジェクトのみ |
| <code>.claude/settings.local.json</code> | ローカル専用（git 管理外） |

3.1.8 🏆 練習問題

1. **【確認】** フックが使える7つのイベントを全て答えてください。
2. **【確認】** PreToolUse フックで exit code 2 を返すと何が起きますか？
3. **【実践】** `~/.claude/settings.json` を開き、現在のフック設定を確認しましょう（なければ空の `hooks: {}` を確認）。
4. **【応用】** ファイル編集後に自動でフォーマットを実行するフックを設定するには、どのイベントとどのマッチャーを使いますか？

3.1.9 次のステップ

→ 3-2: フックの種類 に進む

3.2 3-2: フックの種類

学習時間: 15分 | 難易度: ★★

3.2.1 PreToolUse

ツールが実行される直前に発火します。ツールへの入力は **stdin** から **JSON** として渡されます。ブロックするには `permissionDecision: "deny"` を含む JSON を **stdout** に出力するか、**exit code 2** を返します（exit code 2 の場合は **stderr** メッセージがトランスクリプトに表示されツールはブロックされます）。

3.2.1.1 設定例: rm -rf をブロック

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'CMD=$(cat | jq -r .tool_input.co
mmand); if echo \"$CMD\" | grep -q \"rm -rf\"; then echo \"{\\\"hookSpecificOutput\\\":{\\\"hookEventName\\\":\\\"PreToolUse
\\\",\\\"permissionDecision\\\":\\\"deny\\\",\\\"permissionDecisi
onReason\\\":\\\"rm -rf は危険なためブロックしました\\\"}}\"; fi'"
          }
        ]
      }
    ]
  }
}
```

スクリプトファイルに切り出すと読みやすくなります（`.claude/hooks/block-rm.sh`）：

```
#!/bin/bash
COMMAND=$(cat | jq -r '.tool_input.command // empty')
if echo "$COMMAND" | grep -q 'rm -rf'; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "rm -rf は危険なためブロックしまし
た"
    }
  }'
```

```
fi
```

3.2.1.2 設定例: git push 前に確認

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash(git push*)",
        "hooks": [
          {
            "type": "command",
            "command": "scripts/pre-push-check.sh"
          }
        ]
      }
    ]
  }
}
```

3.2.2 PostToolUse

ツールが実行された直後に発火します。自動フォーマット・テスト実行に使います。

3.2.2.1 設定例: ファイル編集後に Prettier を実行

ツールの入力は stdin から JSON で取得します。Edit/Write ツールのファイルパスは `.tool_input.file_path` に入っています。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'FILE=$(cat | jq -r .tool_input.file_path); npx prettier --write \"$FILE\" 2>/dev/null || true'"
          }
        ]
      }
    ]
  }
}
```

3.2.2.2 設定例: テストファイル変更後に自動テスト

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'FILE=$(cat | jq -r .tool_input.file_path); if echo \"$FILE\" | grep -q \"\\.test\\.\"; then npm test -- --run; fi'"
          }
        ]
      }
    ]
  }
}
```

3.2.3 Stop

Claude Code がレスポンスを完了したとき（ターンが終わったとき）に発火します。

3.2.3.1 設定例: 完了したら通知音を鳴らす

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "afplay /System/Library/Sounds/Glass.aiff"
          }
        ]
      }
    ]
  }
}
```

3.2.3.2 設定例: 完了時に Slack 通知

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "curl -X POST $SLACK_WEBHOOK -d '{\"text\": \"Claude Code がタスクを完了しました\"}'"
          }
        ]
      }
    ]
  }
}
```

3.2.4 Notification

Claude Code が通知を発生させたとき（承認待ち・エラー等）に発火します。

3.2.4.1 設定例: 承認待ちを通知

```
{
  "hooks": {
    "Notification": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"Claude Code が入力待ちです\" with title \"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

3.2.5 UserPromptSubmit

ユーザーがメッセージを送信した直後に発火します。入力の前処理や検証に使用します。

3.2.5.1 設定例: 入力をログに記録

UserPromptSubmit の stdin には `{"prompt": "...", "permission_mode": "..."}` の形式で入力が渡されます。

```
{
  "hooks": {
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'PROMPT=$(cat | jq -r .prompt); echo \"$(date): $PROMPT\" >> ~/.claude/prompt-history.log'"
          }
        ]
      }
    ]
  }
}
```

3.2.6 SubagentStop

サブエージェントが処理を完了したときに発火します。 `matcher` にはエージェントタイプを指定できます。

3.2.6.1 設定例: サブエージェント完了を通知

```
{
  "hooks": {
    "SubagentStop": [
      {
        "matcher": "Explore",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Explore エージェントが完了しました'"
          }
        ]
      }
    ]
  }
}
```

3.2.7 PreCompact

コンテキスト圧縮が実行される直前に発火します。`matcher` には `manual`（手動）または `auto`（自動）を指定できます。`exit code 2` で圧縮をブロックできます。

3.2.7.1 設定例: 圧縮前にログを記録

```
{
  "hooks": {
    "PreCompact": [
      {
        "matcher": "auto",
        "hooks": [
          {
            "type": "command",
            "command": "echo '[$(date)] コンテキスト自動圧縮が実行
されます' >> ~/.claude/compact.log"
          }
        ]
      }
    ]
  }
}
```

3.2.8 matcher の書き方

| パターン | マッチするツール |
|--------------------------------|--|
| <code>"Edit"</code> | Edit ツールのみ |
| <code>"Edit\ Write"</code> | Edit または Write (<code>\ </code> でエスケープして記述) |
| <code>"Bash"</code> | 全 Bash コマンド |
| <code>"Bash(git *)"</code> | git で始まる Bash のみ |
| <code>"mcp__memory__.*"</code> | memory サーバーの全ツール (正規表現) |
| <code>" "</code> または省略 | 全ツール |

英数字・`_`・`|` のみで構成されるパターンはパイプ区切りリストとして扱われます。それ以外の文字が含まれる場合は JavaScript 正規表現として解釈されます。

3.2.9 ツール入力・出力の取得

フックコマンドはツールの入力・出力を `stdin` から `JSON` として受け取ります (環境変数ではありません)。 `jq` を使って値を抽出します：

```
# PreToolUse: ツールへの入力コマンドを取得
COMMAND=$(cat | jq -r '.tool_input.command // empty')

# PostToolUse: 編集されたファイルパスを取得
FILE=$(cat | jq -r '.tool_input.file_path // empty')

# UserPromptSubmit: ユーザーの入力テキストを取得
PROMPT=$(cat | jq -r '.prompt // empty')
```

3.2.10 環境変数

フックコマンド内で利用できるシステム環境変数：

| 変数 | 利用可能なイベント | 内容 |
|---|--------------|--|
| <code>CLAUDE_ENV</code>
<code>_FILE</code> | SessionStart | 環境変数を永続化するためのファイルパス |
| <code>CLAUDE_EFFECT</code> | ツール使用イベント | 努力レベル
(low / medium / high / xhigh / max) |
| <code>CLAUDE_CODE_REMOTE</code> | 全イベント | Web 環境で実行中の場合 <code>"true"</code> |

コマンド文字列内で使えるパスプレースホルダー：

| プレースホルダー | 内容 |
|-------------------------------------|--------------------|
| <code>\${CLAUDE_PROJECT_DIR}</code> | プロジェクトルートの絶対パス |
| <code>\${CLAUDE_PLUGIN_ROOT}</code> | プラグインのインストールディレクトリ |
| <code>\${CLAUDE_PLUGIN_DATA}</code> | プラグインの永続データディレクトリ |

3.2.11 🛠️ 練習問題

1. **【確認】** `PreToolUse` と `PostToolUse` の違いを説明してください。
2. **【実践】** Edit ツールの `PostToolUse` で `echo "file edited"` を実行するフックを設定してみましょう。
3. **【実践】** Stop イベントで通知を出すフックを作成し、動作を確認しましょう。
4. **【応用】** `Notification` フックを使って、モバイルに通知を送るにはどのようなコマンドが必要ですか？

3.2.12 次のステップ

→ 3-3: 自動化パターン に進む

3.3 3-3: 自動化パターン

学習時間: 15分 | 難易度: ★★

3.3.1 パターン1: コード品質の自動維持

ファイルを編集するたびに lint と format を自動実行するパターンです。

ツールの入力は `stdin` から `JSON` で受け取ります。 `jq` でファイルパスを取り出して使います：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'FILE=$(cat | jq -r .tool_input.file_path); npx eslint --fix \"${FILE}\" 2>/dev/null; npx prettier -write \"${FILE}\" 2>/dev/null; true'"
          }
        ]
      }
    ]
  }
}
```

効果: Claude Code がファイルを編集するたびに自動でコードが整形されます。

3.3.2 パターン2: テスト自動実行

コードを変更するたびに関連テストを実行するパターンです。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "scripts/run-related-tests.sh"
          }
        ]
      }
    ]
  }
}
```

```
#!/bin/bash
# scripts/run-related-tests.sh
# stdin から JSON でファイルパスを取得
CHANGED_FILE=$(cat | jq -r '.tool_input.file_path // empty')
TEST_FILE="${CHANGED_FILE/src\\//src\\/}".test.ts

if [ -f "$TEST_FILE" ]; then
  npx vitest run "$TEST_FILE" 2>&1
  exit $?
fi
```

3.3.3 パターン3: Git 操作のガード

force push や main への直接 push をブロックするパターンです。

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "scripts/git-guard.sh"
          }
        ]
      }
    ]
  }
}
```

```
#!/bin/bash
# scripts/git-guard.sh
# stdin から JSON でコマンドを取得
INPUT=$(cat | jq -r '.tool_input.command // empty')

# force push をブロック
if echo "$INPUT" | grep -q "push.*--force\|push.*-f"; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "force push は禁止されています。レビューを経てください。"
    }
  }'
  exit 0
fi

# main への直接 push をブロック
if echo "$INPUT" | grep -qE "push.*origin main\|push.*origin master"; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "main への直接 push は禁止されています。PR を作成してください。"
    }
  }'
  exit 0
fi
```

3.3.4 パターン4: 完了通知

長時間のタスクが完了したときに通知するパターンです。

3.3.4.1 macOS の場合

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"タスクが完了しました\" with title \"Claude Code\" sound name \"Glass\"'"
          }
        ]
      }
    ]
  }
}
```

3.3.4.2 Windows の場合

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "powershell -command \"[System.Console]::Beep(1000, 300)\""
          }
        ]
      }
    ]
  }
}
```

3.3.5 パターン5: 作業ログの記録

すべての Bash コマンドと結果をログに記録するパターンです。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'CMD=$(cat | jq -r .tool_input.co
mmand); echo \"[${date +%Y-%m-%d\\ %H:%M:%S}] CMD: $CMD\" >> ~/.c
laude/activity.log'"
          }
        ]
      }
    ]
  }
}
```

3.3.6 パターン6: 型チェックの自動実行

TypeScript ファイルを編集するたびに型チェックを実行するパターンです。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'FILE=$(cat | jq -r .tool_input.f
ile_path); if echo \"$FILE\" | grep -q '\\.tsx\\|$\"; then npx t
sc --noEmit 2>&1 | tail -20; fi'"
          }
        ]
      }
    ]
  }
}
```

3.3.7 複数フックの組み合わせ

同じイベントに複数のフックを登録できます：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "bash -c 'FILE=$(cat |
jq -r .tool_input.file_path); npx prettier --write \"${FILE}\" 2>/d
ev/null || true' " },
          { "type": "command", "command": "bash -c 'FILE=$(cat |
jq -r .tool_input.file_path); npx eslint --fix \"${FILE}\" 2>/dev/n
ull || true' " }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          { "type": "command", "command": "afplay /System/Librar
y/Sounds/Glass.aiff" }
        ]
      }
    ]
  }
}
```

3.3.8 🍷 練習問題

1. **【確認】** フックを使って lint エラーがある場合にファイル編集をブロックするには、どのフックタイプを使いますか？
2. **【実践】** ファイル編集後に `echo "編集完了: $FILE"` を出力するフックを設定してみましょう（`FILE` は stdin JSON から `jq` で取得します）。

3. 【実践】 Claude が完了した際に OS 通知（macOS: `osascript`、Windows: `powershell -command "[System.Console]::Beep(1000, 300)"`）を送るフックを作成しましょう。
4. 【応用】 CI/CD 環境でフックを使う場合の注意点を2つ挙げてください。

3.3.9 次のステップ

→ 3-4: settings.json 設定 に進む

3.4 3-4: settings.json 設定

学習時間: 15分 | 難易度: ★★

3.4.1 settings.json の場所と優先順位

| ファイル | 適用範囲 | 用途 |
|--|------------|------------------|
| マネージドポリシー設定 | 組織全体 | IT 管理者が展開（上書き不可） |
| <code>~/.claude/settings.json</code> | グローバル | 全プロジェクト共通の個人設定 |
| <code>.claude/settings.json</code> | プロジェクト | チームで共有する設定 |
| <code>.claude/settings.local.json</code> | プロジェクトローカル | 個人設定（git 管理外） |

優先順位（高い順）：マネージドポリシー > コマンドライン引数 > ローカル設定 > プロジェクト設定 > ユーザー設定

なお、permissions ルールはスコープをまたいでマージされます（上書きではありません）。

3.4.2 完全な設定例

```
{
  "model": "claude-sonnet-4-6",
  "theme": "dark",
  "permissions": {
    "allow": [
      "Bash(git *)",
      "Bash(npm run *)",
      "Bash(npm test)",
      "Bash(npx prettier *)",
      "Bash(npx eslint *)",
      "Read(**)",
      "Edit(**)",
      "Write(**)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(DROP TABLE *)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'FILE=$(cat | jq -r .tool_input.file_path); npx prettier --write \"$FILE\" 2>/dev/null || true'"
          }
        ]
      }
    ]
  }
}
```

```
    }  
  ],  
  "Stop": [  
    {  
      "hooks": [  
        {  
          "type": "command",  
          "command": "echo 'Task completed'"  
        }  
      ]  
    }  
  ],  
},  
"env": {  
  "NODE_ENV": "development",  
  "LOG_LEVEL": "debug"  
},  
"mcpServers": {  
  "filesystem": {  
    "command": "npx",  
    "args": ["-y", "@modelcontextprotocol/server-filesystem",  
"/Users/me/projects"]  
  }  
}  
}
```

3.4.3 各設定項目の説明

3.4.3.1 model

使用するモデルを指定します：

```
{
  "model": "claude-opus-4-6"
}
```

| モデル | 特徴 |
|-------------------|--------------|
| claude-opus-4-6 | 最高性能 |
| claude-sonnet-4-6 | バランス型（デフォルト） |
| claude-haiku-4-5 | 高速・低コスト |

3.4.3.2 permissions

ツールの自動承認・拒否・確認要求を設定します：

```
{
  "permissions": {
    "allow": ["Bash(git *)", "Read(**)"],
    "deny": ["Bash(rm -rf *)"],
    "ask": ["Bash(npm publish *)"]
  }
}
```

パターン書式:

| パターン例 | 意味 |
|-----------------------|---------------------|
| "Bash(git *)" | git で始まる全 Bash コマンド |
| "Bash(npm run build)" | 特定のコマンドのみ |
| "Read(**)" | 全ファイルの読み取り |
| "Edit(src/**)" | src/ 以下のファイル編集のみ |

3.4.3.3 env

セッション中に使用する環境変数を設定します：

```
{
  "env": {
    "DATABASE_URL": "postgresql://localhost/mydb",
    "API_BASE_URL": "http://localhost:3001"
  }
}
```

3.4.3.4 theme

UI テーマを設定します：

```
{
  "theme": "dark"
}
```

選択肢: "dark" / "light" / "auto"

3.4.4 プロジェクト用 settings.json の例

チームで共有する最小設定：

```
{
  "permissions": {
    "allow": [
      "Bash(npm *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git add *)",
      "Bash(git commit *)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "bash -c 'FILE=$(cat |
jq -r .tool_input.file_path); npm run format -- \"$FILE\" 2>/dev/
null || true'" }
        ]
      }
    ]
  }
}
```

3.4.5 settings.local.json の活用

個人の API キーやパスなど、チームと共有しない設定は

`settings.local.json` に書きます：

```
{
  "env": {
    "ANTHROPIC_API_KEY": "sk-ant-...",
    "PERSONAL_ACCESS_TOKEN": "ghp_..."
  },
  "hooks": {
    "Stop": [
      {
        "hooks": [
          { "type": "command", "command": "afplay /System/Library/Sounds/Glass.aiff" }
        ]
      }
    ]
  }
}
```

3.4.6 /config コマンドで設定する

`settings.json` を手書きせず、対話的に設定することもできます：

```
/config
```

3.4.7 フックの一時無効化

全フックを一時的に無効化するには `disableAllHooks` を使います：

```
{
  "disableAllHooks": true
}
```

3.4.8 🏆 練習問題

1. 【確認】 `~/.claude/settings.json` と `.claude/settings.json` のどちらが優先されますか？
2. 【実践】 `permissions.allow` に `["Bash(git *)"]` を追加して、git コマンドを確認なしで実行できるようにしましょう。
3. 【実践】 `model` フィールドでデフォルトモデルを変更してみましょう。
4. 【応用】 チームで共有する `.claude/settings.json` に含めるべき設定と、個人の `settings.local.json` に留めるべき設定の違いを説明してください。

3.4.9 次のステップ

→ 4-1: メモリの種類 に進む

4. メモリシステム

4.1 4-1: メモリの種類

学習時間: 15分 | 難易度: ★★

4.1.1 メモリシステムとは

Claude Code のメモリシステムは、セッションをまたいで情報を保持するための仕組みです。会話が終わっても、重要な情報を次のセッションで利用できます。

Claude Code には 2 種類の独立したメモリ機構があります：

| 機構 | 誰が書くか | 内容 | 読み込みタイミング |
|----------------|--------|---------|------------------------------------|
| CLAUDE.md ファイル | あなた | 指示・ルール | 毎セッション（全文） |
| オートメモリ | Claude | 学習・パターン | 毎セッション（MEMORY.md 先頭 200 行または 25KB） |

4.1.2 CLAUDE.md ファイル

あなたが書いてプロジェクトや作業スタイルを伝えるための指示書です。セッション開始時に毎回読み込まれます。詳細は 4-2: CLAUDE.md ファイル で解説します。

4.1.3 オートメモリ (Auto Memory)

Claude が自分でノートを書く仕組みです。ビルドコマンド・デバッグ知見・アーキテクチャメモ・コードスタイルの好みなどを、Claude が判断して保存します。

バージョン要件: オートメモリは Claude Code v2.1.59 以降が必要です。

`claude --version` で確認してください。

4.1.3.1 保存される情報の例

Claude がオートメモリに保存するのは、将来のセッションで役立つと判断した情報です：

あなた：テストフレームワークは Vitest に統一することになりました

Claude：了解しました。メモリに保存しておきます。

[memory/testing.md を作成]

あなた：PR は必ず 1 機能ずつ分けてレビューしてもらう方針で

Claude：保存しました。

[memory/project-conventions.md を更新]

4.1.3.2 メモリが保存する情報のカテゴリ

Claude はオートメモリを以下のカテゴリで整理することがあります（これは Claude が使う分類であり、固定の仕様ではありません）：

ユーザー情報（あなたの役割・技術レベル・好み）

フルスタック開発者。フロントエンドは React/TypeScript が得意、バックエンドは Go を使用。Docker は使いこなしているが、Kubernetes は学習中。

フィードバック（Claude への修正・確認済みのアプローチ）

レスポンス末尾に「～を行いました」というサマリーを書かない。
ユーザーは diff を直接確認できるので不要。

****Why:**** ユーザーが「末尾のサマリーは不要」と明示的に指示。

****How to apply:**** 全レスポンスで適用。コード変更後のまとめも省略。

プロジェクト情報（進行中の作業・目標・決定事項）

認証モジュールを JWT から Session-based に移行中。

期限：2026-06-30。

****Why:**** セキュリティ監査でセッション管理の不備が指摘された。

****How to apply:**** 認証関連の変更提案はセッション方式を前提にする。

参照情報（外部ツールのリンクなど）

バグは Linear の "Frontend" プロジェクトで管理。

デザインは Figma の "Product v3" ファイル。

4.1.4 メモリの保存場所

```
~/ .claude/projects/<project-hash>/memory/  
├─ MEMORY.md          # インデックス（毎セッション読み込まれる）  
├─ testing.md          # テストに関するメモリ  
├─ project-auth.md     # プロジェクト固有のメモリ  
└─ api-conventions.md  # API 設計の決定事項
```

`<project-hash>` は git リポジトリから導出されます。同じリポジトリのすべての worktree とサブディレクトリが 1 つのメモリディレクトリを共有します。

`MEMORY.md` はインデックスとして機能し、各セッション開始時に先頭 200 行（または 25KB）が読み込まれます。詳細なメモはトピック別ファイルに分けて保存されます。

4.1.5 オートメモリの操作

4.1.5.1 確認する

> 今まで何を覚えているか教えて

または `/memory` コマンドでメモリファイルの一覧を確認できます。

4.1.5.2 保存を依頼する

> この設定を覚えておいて：テストは必ず Vitest を使うこと

4.1.5.3 削除・更新を依頼する

> auth-rewrite プロジェクトのメモリは完了したので削除して

4.1.5.4 オートメモリの有効/無効

オートメモリはデフォルトで有効です。無効にするには `/memory` コマンドのトグルを使うか、設定ファイルに記述します：

```
{  
  "autoMemoryEnabled": false  
}
```

4.1.6 🏆 練習問題

1. **【確認】** Claude Code の 2 種類のメモリ機構（CLAUDE.md とオートメモリ）の違いを、「誰が書くか」と「何を保存するか」の観点から説明してください。
2. **【確認】** オートメモリの `MEMORY.md` が毎セッション読み込まれる上限（行数またはサイズ）を教えてください。
3. **【実践】** `~/.claude/projects/` ディレクトリを確認し、既存のメモリファイルを探してみましょう。
4. **【応用】** コードベースの構造やファイルパスをメモリに保存すべきでない理由を説明してください。

4.1.7 次のステップ

→ 4-2: CLAUDE.md ファイル に進む

4.2 4-2: CLAUDE.md ファイル

学習時間: 15分 | 難易度: ★★

4.2.1 CLAUDE.md とは

`CLAUDE.md` はプロジェクトや個人の指示をまとめるファイルです。Claude Code はセッション開始時（および `/clear` 後）に自動的に読み込みます。

CLAUDE.md に書いたことは、毎回会話で説明しなくても常に有効です。

4.2.2 CLAUDE.md を置く場所

CLAUDE.md は複数の場所に置けます。スコープが広い順に読み込まれます：

| スコープ | 場所 | 用途 | 共有範囲 |
|----------|--|----------------|---------------------|
| 管理ポリシー | macOS: <code>/Library/Application Support/ClaudeCode/CLAUDE.md</code>
Linux/WSL: <code>/etc/claude-code/CLAUDE.md</code>
Windows: <code>C:\Program Files\ClaudeCode\CLAUDE.md</code> | 組織全体の指示（IT 管理） | 全ユーザー |
| ユーザー指示 | <code>~/.claude/CLAUDE.md</code> | 全プロジェクト共通の個人設定 | 自分のみ（全プロジェクト） |
| プロジェクト指示 | <code>./CLAUDE.md</code> または <code>./.claude/CLAUDE.md</code> | チーム共有のプロジェクト設定 | チーム全員（バージョン管理） |
| ローカル指示 | <code>./CLAUDE.local.md</code> | 個人のプロジェクト固有設定 | 自分のみ（.gitignore 推奨） |

4.2.2.1 階層配置

複数の CLAUDE.md を階層的に配置できます：

```
my-project/
├─ CLAUDE.md          # プロジェクト全体の指示
├─ .claude/
│   └─ rules/         # パス別ルール（後述）
├─ src/
│   └─ CLAUDE.md      # src/ 以下のファイル进行操作するときに追加読み込み
└─ docs/
    └─ CLAUDE.md      # docs/ 以下のファイル进行操作するときに追加読み込み
```

プロジェクトルートより上のディレクトリの CLAUDE.md は起動時に全文読み込まれます。サブディレクトリの CLAUDE.md は、そのディレクトリのファイルを Claude が操作するときに読み込まれます。

4.2.3 自動生成

```
claude
/init
```

Claude Code がプロジェクトを分析して CLAUDE.md の初稿を自動生成します。すでに CLAUDE.md がある場合は、改善案を提案するだけで上書きしません。生成後、手動で調整します。

4.2.4 推奨構造

プロジェクト名

概要

このプロジェクトの1~2行の説明。

技術スタック

- 言語: TypeScript 5.x
- フレームワーク: React 18
- テスト: Vitest + Testing Library
- スタイル: Tailwind CSS
- ビルド: Vite

よく使うコマンド

- `npm run dev` - 開発サーバー起動 (ポート 3000)
- `npm test` - テスト実行 (watch モード)
- `npm run build` - プロダクションビルド
- `npm run lint` - ESLint チェック
- `npm run format` - Prettier フォーマット

ディレクトリ構造

src/

- ├─ components/ # UI コンポーネント
- ├─ hooks/ # カスタムフック
- ├─ utils/ # ユーティリティ関数
- ├─ types/ # TypeScript 型定義
- └─ api/ # API クライアント

コーディング規約

- コンポーネントは関数コンポーネントのみ (クラスコンポーネント禁止)
- Props の型は interface で定義 (type alias 禁止)
- テストファイルはコンポーネントと同じディレクトリに配置

- インポートは絶対パス（`@/` エイリアス）を使用

注意事項

- `src/legacy/` は触らない（リプレイス予定）
- 環境変数は `.env.local` に書く（`.env` は git 管理）
- API モックは `msw` を使用（`fetch` のモックは禁止）

4.2.5 ファイルのインポート（@ 構文）

CLAUDE.md から他のファイルを @パス 構文でインポートできます：

プロジェクト概要は @README を参照。npm コマンドは @package.json を参照。

追加指示

- git ワークフロー @docs/git-instructions.md

インポートされたファイルは起動時にコンテキストへ展開されます。相対パス・絶対パスどちらも使用可能です。パスをバッククォートで囲む（`@README`）とインポートされずリテラルとして扱われます。

4.2.6 .claude/rules/ によるルール整理

大規模プロジェクトでは `.claude/rules/` ディレクトリに複数のルールファイルを配置できます：

```
your-project/
├─ CLAUDE.md
└─ .claude/
    └─ rules/
        ├─ code-style.md    # コードスタイルガイドライン
        ├─ testing.md       # テスト規約
        └─ security.md      # セキュリティ要件
```

4.2.6.1 パス別ルール

YAML フロントマターで `paths` フィールドを指定すると、特定のファイル进行操作するときだけルールを読み込みます：

```
---
paths:
  - "src/api/**/*.ts"
---
```

API 開発ルール

- 全 API エンドポイントは入力バリデーションを含めること
- 標準エラーレスポンス形式を使用
- OpenAPI ドキュメントコメントを含めること

4.2.7 書くべき内容・書かない内容

| 書くべき | 書かない |
|--------------|--------------------------------------|
| 技術スタックとバージョン | コードの実装詳細（コードを読めばわかる） |
| よく使うコマンド | git 履歴（ <code>git log</code> で確認できる） |
| コーディング規約 | 一時的なタスクの状態 |
| 触ってはいけないファイル | 汎用的な開発常識 |
| 外部ツールの接続情報 | 既に CLAUDE.md に書いてある内容の繰り返し |

サイズの目安: 1 ファイルあたり 200 行以内を推奨。長いファイルはコンテキストを消費し、指示の遵守率が下がります。

4.2.8 効果的な書き方のコツ

4.2.8.1 理由を添える

悪い例

- テストには Vitest を使う

良い例

- テストには Vitest を使う（Jest から移行済み。新しく Jest を追加しないこと）

4.2.8.2 禁止事項は明確に

禁止事項

- `any` 型の使用（TypeScript の型安全性を損なう）
- `console.log` をコミットすること（デバッグ用のみ）
- `src/legacy/` の直接編集（移行チームの担当）

4.2.8.3 具体的に書く

| 曖昧（悪い） | 具体的（良い） |
|-----------------|--|
| コードを正しくフォーマットする | インデントは 2 スペースを使う |
| 変更前にテストする | <code>npm test</code> を実行してからコミットする |
| ファイルを整理する | API ハンドラは <code>src/api/handlers/</code> に配置する |

4.2.9 `/memory` コマンドで確認

`/memory` コマンドを使うと、現在のセッションで読み込まれているすべての `CLAUDE.md`・`CLAUDE.local.md`・ルールファイルの一覧を確認できます。ファイルが読み込まれていない場合は Claude に見えていません。

4.2.10 🛠️ 練習問題

1. **【確認】** `CLAUDE.md` はいつ Claude Code に読み込まれますか？ また、サブディレクトリの `CLAUDE.md` が読み込まれるタイミングはいつですか？
2. **【実践】** `/init` を使って現在のプロジェクトの `CLAUDE.md` を生成し、内容を確認しましょう。
3. **【実践】** `CLAUDE.md` に「このプロジェクトでは `console.log` をコミットしないこと」を追記して、次のセッションで有効になるか確認しましょう。

4.【応用】 `src/` と `docs/` でそれぞれ異なる CLAUDE.md を配置するのが有効な場面を考えてみましょう。

4.2.11 次のステップ

→ 4-3: 永続的なメモリ に進む

4.3 4-3: 永続的なメモリ（オートメモリ）

学習時間: 15分 | 難易度: ★★

4.3.1 セッションをまたいだ記憶

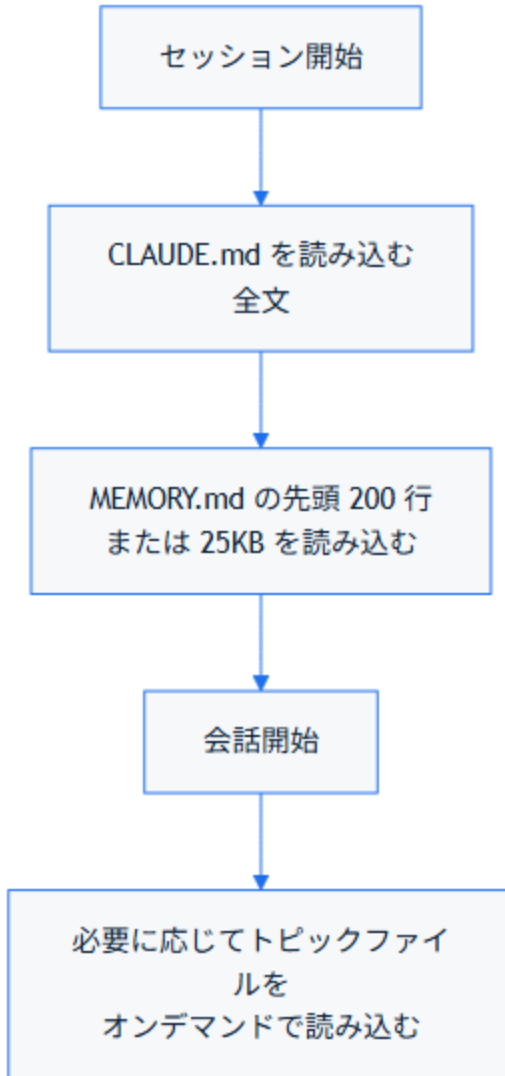
Claude Code のオートメモリシステムは

`~/.claude/projects/<project>/memory/` にファイルとして保存されます。新しいセッションを開始したときや `/clear` 後でも、このメモリは引き継がれます。

`<project>` は git リポジトリから導出されます。同じリポジトリのすべての `worktree` とサブディレクトリが 1 つのメモリディレクトリを共有します（git リポジトリ外ではプロジェクトルートを使用）。

注意: オートメモリはマシンローカルです。他のマシンやクラウド環境とは共有されません。

4.3.2 メモリの読み込みタイミング



`MEMORY.md` はインデックスとして機能します。詳細なメモは `debugging.md` や `api-conventions.md` などのトピックファイルに分けて保存され、Claude が必要と判断したときに読み込まれます。

4.3.3 メモリの保存・更新

4.3.3.1 自動保存

Claude Code が重要な情報を検出すると自動的にメモリを作成します：

> テストフレームワークは Vitest に統一することになりました

Claude: 了解しました。メモリに保存しておきます。

[memory/testing.md を作成]

4.3.3.2 手動で保存を依頼

> 「PR は必ず 1 機能ずつ分けてレビューしてもらう」という方針を覚えておいて

Claude: 保存しました。

[memory/project-conventions.md を作成]

4.3.4 MEMORY.md インデックス

各メモリへのポインタとなるインデックスファイルです。セッション開始時に先頭 200 行（または 25KB）が読み込まれます：

Memory Index

ユーザー

- [user-role](user-role.md) - フルスタック開発者、React/TypeScript
+ Go 専門、Docker、Kubernetes 学習中

フィードバック

- [feedback-no-trailing-summary](feedback-no-trailing-summary.md)
- レスポンス末尾のサマリー不要 (diff を直接確認するため)
- [feedback-testing](feedback-testing.md) - テストフレームワークは Vitest のみ使用、Jest 禁止

プロジェクト

- [project-auth](project-auth.md) - 認証モジュール JWT→Session 移行中 (期限: 2026-06-30)
- [project-freeze](project-freeze.md) - 2026-06-20 以降マージ凍結 (モバイルリリース対応)

参照

- [reference-design](reference-design.md) - Figma "Product v3" ファイル、Linear "Frontend" プロジェクト

4.3.4.1 対応するトピックファイルのサンプル

MEMORY.md の各行が指すファイルはこのような内容になります：

feedback-testing.md

テストは必ず Vitest を使う。Jest は使わない。

Why: 昨年 Jest のモックが本番と挙動が異なり、リリース後に障害が発生した。

How to apply: テストコードを書くとき・修正するとき全般に適用。

project-auth.md

認証モジュールを JWT から Session-based に移行中。期限: 2026-06-30。

****Why:**** セキュリティ監査でセッション管理の不備が指摘された（コンプライアンス要件）。

****How to apply:**** 認証関連の変更はセッション方式を前提に提案する。エルゴノミクスより準拠を優先。

トピックファイルはシンプルな Markdown です。フロントマターは必須ではありません（Claude が自由にフォーマットを決めます）。

4.3.5 /memory コマンド

/memory コマンドでメモリを管理できます：

- 現在のセッションで読み込まれた全 CLAUDE.md・ルールファイルの一覧表示
- オートメモリのオン/オフ切り替え
- メモリフォルダを開くリンクの表示

4.3.6 メモリの確認

> 今どんなことを覚えているか教えて

> feedback に関するメモリを全部見せて

4.3.7 メモリの更新

> auth プロジェクトの期限が 7 月末に変更になったので更新して

4.3.8 メモリの削除

> project-auth のメモリは完了したので削除して

4.3.9 メモリとして保存すべき情報

| 保存する | 保存しない |
|--------------|-------------------|
| ユーザーの技術的背景 | 実装の詳細（コードを見ればわかる） |
| 繰り返し出てきた要望 | 一時的なデバッグの結果 |
| プロジェクトの方針・制約 | 汎用的な技術情報 |
| 外部ツールのリンク | 現在の会話コンテキスト |
| 確認済みのアプローチ | git 履歴で追える情報 |

4.3.10 メモリファイルの手動編集

メモリファイルはただの Markdown ファイルなので、直接編集できます：

```
code ~/.claude/projects/<hash>/memory/feedback-testing.md
```

4.3.11 メモリの有効期限に注意

メモリは「書いたときに正しかった情報」です。コードやプロジェクトが変わっても、メモリは自動更新されません。定期的に見直して古くなったメモリを削除・更新しましょう。

> メモリの内容を確認して、現在の状況と合っているか確認して

4.3.12 🏆 練習問題

1. **【確認】** オートメモリの保存パスを教えてください。また、`MEMORY.md` が毎セッション読み込まれる上限（行数・サイズ）は何ですか？
2. **【実践】** `user` に関する情報（例：自分の技術スタック）を Claude に伝え、メモリに保存してもらいましょう。
3. **【実践】** 新しいセッションを開始し、前のセッションで保存したメモリが引き継がれているか確認しましょう。
4. **【応用】** メモリが古くなったとき（例：使っていたライブラリを変更したとき）はどうすべきですか？

4.3.13 次のステップ

→ 4-4: メモリのベストプラクティス に進む

4.4 4-4: メモリのベストプラクティス

学習時間: 15分 | 難易度: ★★

4.4.1 メモリ設計の原則

4.4.1.1 原則1: 「なぜ」を必ず記録する

ルールだけを書いても、例外ケースで正しく適用できません：

悪い例

テストは必ず Vitest を使う。

良い例

テストは必ず Vitest を使う。

****Why:**** Jest から Vitest に移行済み。Jest の設定ファイルは残っているが使わない。

****How to apply:**** 新しいテストファイルを作るとき、package.json を変更するとき。

****Why:**** にはルールができた背景・理由を書きます。 ****How to apply:**** にはどの場面でどう適用するかを書きます。この 2 行があることで、Claude が文脈に応じて正しく判断できます。

4.4.1.2 原則2: 適切な粒度で分割する

1 つのメモリファイルに詰め込みすぎない：

悪い例：1つのファイルに全部

memory/everything.md

良い例：トピックごとに分割

memory/feedback-testing.md

memory/feedback-pr-policy.md

memory/project-auth-rewrite.md

memory/reference-design-tools.md

4.4.1.3 原則3: 定期的に棚卸する

月に1回、メモリの内容が現状に合っているか確認します：

＞ メモリの一覧を見せて。古くなっているものがあれば教えて

4.4.1.4 原則4: コードで追える情報は書かない

以下は CLAUDE.md やメモリに書く必要がありません： - どのファイルに何が書いてあるか（`grep` でわかる） - 最近の変更内容（`git log` でわかる） - コードの実装パターン（コードを読めばわかる） - コードベースの具体的なディレクトリ構造（`ls` や `find` でわかる）

4.4.2 フィードバックメモリの書き方

フィードバックは修正（×したこと）だけでなく、承認（✓したこと）も記録します：

大規模リファクタリングは分割せず1つの PR にまとめる。

****Why:**** 小さく分割すると diff が追いにくくなることがユーザーが確認した。

****How to apply:**** リファクタリング系のタスクで PR 分割を提案しない。

4.4.3 CLAUDE.md とオートメモリの使い分け

| 情報の種類 | 置き場所 |
|---------------------|-----------------------|
| プロジェクト全体の技術スタック | CLAUDE.md |
| よく使うコマンド | CLAUDE.md |
| コーディング規約 | CLAUDE.md |
| チームの規約・ポリシー | CLAUDE.md（バージョン管理で共有） |
| ユーザーの技術的背景・好み | オートメモリ |
| Claude への修正・フィードバック | オートメモリ |
| 進行中プロジェクトの状況 | オートメモリ |
| 外部ツールのリンク | オートメモリ |

個人の作業スタイルや好み → オートメモリ、チームの規約 → CLAUDE.md に分離するのが基本原則です。

4.4.4 CLAUDE.md とオートメモリの ToDo との違い

| ツール | 用途 |
|-----------|------------------------------|
| CLAUDE.md | 恒久的な指示・ルール（常にセッションに持ち込む） |
| オートメモリ | Claude が自分で気づいた学習・パターン |
| TodoWrite | 現在のセッション内のタスク追跡（セッション終了後は不要） |

一時的なタスクの進捗は TodoWrite で管理し、永続的なルールは CLAUDE.md、セッションをまたぐ学習はオートメモリ、という使い分けが有効です。

4.4.5 メモリが肥大化した場合

`MEMORY.md` の先頭 200 行（または 25KB）がセッション開始時に読み込まれます。これを超えると読み込まれないため、不要なメモリを整理するか、複数のメモリをマージします：

> メモリが増えすぎているので、feedback 関連のものをまとめて整理して

詳細な情報はトピックファイル（`debugging.md` など）に移して、`MEMORY.md` には簡潔な参照だけを残しましょう。

4.4.6 メモリのサイズと件数の管理

- `MEMORY.md` は 200 行以内を目安に保つ
- トピックファイルには詳細を書いても OK（オンデマンドで読み込まれる）

- 古いメモリは定期的に削除または更新する
- 1 ファイルには 1 トピック（関連しない情報を混在させない）

4.4.7 チームでのメモリ活用

オートメモリはユーザーごと・マシンごとに独立しているため、チームメンバーと共有できません。

チーム共通の知識は CLAUDE.md に書きましょう：

- チームの規約・スタイルガイド → プロジェクトの CLAUDE.md に記述（git で共有）
- 個人の作業スタイル・好み → オートメモリに記録
- 組織全体のポリシー → 管理ポリシー CLAUDE.md（IT 部門が配布）

チームメンバー間でメモリを共有したい場合は、その内容を CLAUDE.md に移すか、`.claude/rules/` のルールファイルとして管理するのが適切です。

4.4.8 🏋️ 練習問題

1. **【確認】** コードパターン・アーキテクチャをメモリに保存しない理由を説明してください。
2. **【確認】** フィードバックメモリの `**Why:**` と `**How to apply:**` 行の役割を説明してください。
3. **【実践】** 過去の作業で「次回のセッションでも覚えておいてほしいこと」を 1 件メモリに保存してみましょう。
4. **【応用】** チームメンバー間でメモリを共有したい場合、どのような方法が考えられますか？

4.4.9 次のステップ

→ 5-1: MCP の概要 に進む

5. MCP とエージェント

5.1 5-1: MCP の概要

学習時間: 15分 | 難易度: ★★ ★

5.1.1 MCP（Model Context Protocol）とは

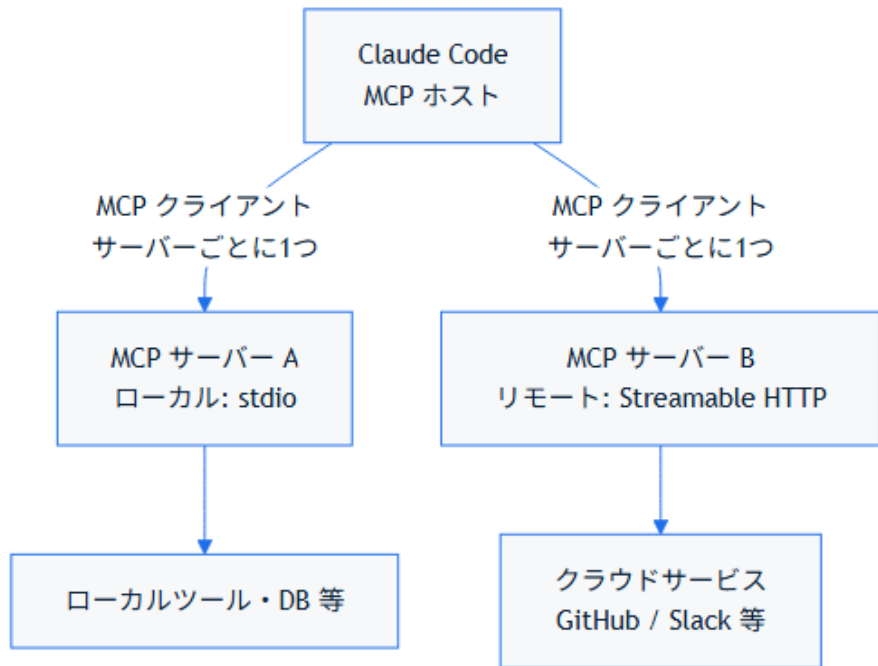
MCP は Anthropic が策定したオープン標準プロトコルです。AI モデルと外部ツール・データソースを安全に接続するための共通インターフェースを定義しています。

Claude Code は MCP クライアントとして動作し、MCP サーバーを通じて外部システムと連携できます。

5.1.2 MCP でできること

| カテゴリ | 例 |
|-----------|---------------------------|
| ファイルシステム | 指定ディレクトリの読み書き |
| データベース | SQLite / PostgreSQL クエリ |
| バージョン管理 | GitHub Issue / PR 操作 |
| Web | URL フェッチ、検索 |
| コミュニケーション | Slack メッセージ送信 |
| クラウド | AWS / GCP / Azure 操作 |
| 開発ツール | Jira / Linear / Notion 操作 |

5.1.3 MCP のアーキテクチャ



MCP は2層で構成されています：

- **データ層:** JSON-RPC 2.0 ベースのプロトコル。Tools / Resources / Prompts などのプリミティブを定義
- **トランスポート層:** 通信チャネル。stdio（ローカル）と Streamable HTTP（リモート）の2種類

5.1.3.1 トランスポートの種類

| トランスポート | 用途 | 特徴 |
|-----------------|-----------------|-----------------------------------|
| stdio | ローカルプロセス
間通信 | 低レイテンシ、ネ
ットワーク不要 |
| Streamable HTTP | リモートサーバー
接続 | OAuth 2.0 認
証対応、クラウド
サービス向け |
| SSE（非推奨） | リモートサーバー
接続 | 後方互換のため
に残存、新規利
用は非推奨 |

5.1.4 MCP が提供する3種類の機能

5.1.4.1 Tools（ツール）

Claude Code が呼び出せる関数。ファイル読み書き、API 呼び出しなど。

5.1.4.2 Resources（リソース）

Claude Code がコンテキストとして読み込めるデータ。ドキュメント、DB レコードなど。`@サーバー名:プロトコル://パス` の形式でプロンプト内から参照できます。

@github:issue://123 の内容を確認して修正案を提案して

5.1.4.3 Prompts（プロンプト）

再利用可能なプロンプトテンプレート。`/mcp__サーバー名__プロンプト名` の形式でスラッシュコマンドとして実行できます。

5.1.5 MCP vs 組み込みツール

| 観点 | 組み込みツール（Read/Edit/Bash 等） | MCP ツール |
|------|---------------------------|---|
| 目的 | ローカルファイル・シェル操作 | 外部サービス連携 |
| 設定 | 不要 | <code>claude mcp add</code> コマンドまたは設定ファイル |
| 追加方法 | できない | MCP サーバーをインストール |
| 認証 | 不要 | API キー・OAuth 等が必要 |

5.1.6 公式 MCP サーバー一覧

modelcontextprotocol/servers や Anthropic Directory で多数の公式サーバーが公開されています：

| サーバー | 用途 |
|---|---------------|
| @modelcontextprotocol/server-filesystem | ファイルシステム操作 |
| @modelcontextprotocol/server-github | GitHub 操作 |
| @modelcontextprotocol/server-sqlite | SQLite DB 操作 |
| @modelcontextprotocol/server-brave-search | Web 検索 |
| @modelcontextprotocol/server-slack | Slack 操作 |
| @modelcontextprotocol/server-postgres | PostgreSQL 操作 |

Anthropic Directory に登録されているリモートサーバーは `claude mcp add` コマンドで直接追加できます。

5.1.7 🏆 練習問題

1. **【確認】** MCP が提供する3種類の機能（Tools / Resources / Prompts）をそれぞれ説明してください。
2. **【確認】** 組み込みツール（Read/Edit/Bash）と MCP ツールの主な違いを2つ答えてください。
3. **【実践】** 公式の MCP サーバー一覧（modelcontextprotocol/servers または Anthropic Directory）を確認し、自分のプロジェクトで使えそうなサーバーを1つ選んでみましょう。

4. 【応用】GitHub MCP サーバーを使うことで、現在の作業フローのどの部分を改善できますか？

5.1.8 次のステップ

→ 5-2: MCP サーバー設定 に進む

5.2 5-2: MCP サーバー設定

学習時間: 20分 | 難易度: ★★ ★

5.2.1 設定方法

MCP サーバーは CLI コマンド（`claude mcp add`）または設定ファイルの直接編集で追加できます。

5.2.2 `claude mcp add` コマンド（推奨）

HTTP サーバー（リモート）を追加

```
claude mcp add --transport http <名前> <URL>
```

stdio サーバー（ローカル）を追加

```
claude mcp add --transport stdio <名前> -- <コマンド> [引数...]
```

環境変数付きで追加

```
claude mcp add --env KEY=value --transport stdio <名前> -- <コマンド>
```

スコープを指定して追加（local/project/user）

```
claude mcp add --scope project --transport http <名前> <URL>
```

5.2.2.1 サーバーの管理コマンド

```
# 設定済みサーバーの一覧表示
claude mcp list

# 特定サーバーの詳細表示
claude mcp get github

# サーバーの削除
claude mcp remove github

# OAuth 認証 (HTTP サーバー)
claude mcp login sentry

# (Claude Code セッション内) サーバー状態確認
/mcp
```

5.2.3 設定スコープ

MCP サーバーの設定は3つのスコープで管理されます：

| スコープ | 有効範囲 | チームへの共有 | 保存先 |
|---------------|-------------|--------------|-----------------|
| local (デフォルト) | 現在のプロジェクトのみ | なし (非公開) | ~/ .claude.json |
| project | 現在のプロジェクトのみ | あり (バージョン管理) | .mcp.json |
| user | 全プロジェクト | なし (非公開) | ~/ .claude.json |

5.2.4 基本的な設定例

5.2.4.1 HTTP サーバー（Streamable HTTP / リモート）

```
# コマンドで追加
claude mcp add --transport http notion https://mcp.notion.com/mcp

# 認証ヘッダー付き
claude mcp add --transport http github https://api.githubcopilot.
com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"
```

`.mcp.json` に直接書く場合：

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": {
        "Authorization": "Bearer YOUR_GITHUB_PAT"
      }
    }
  }
}
```

5.2.4.2 stdio サーバー（ローカルプロセス）

```
# コマンドで追加
claude mcp add --transport stdio filesystem -- npx -y @modelconte
xtprotocol/server-filesystem /Users/me/projects
```

`.mcp.json` に直接書く場合：

```
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/me/projects"
      ]
    }
  }
}
```

5.2.4.3 環境変数による API キー渡し

```
# コマンドで追加 (--env で環境変数を渡す)
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio
airtable \
  -- npx -y airtable-mcp-server
```

`.mcp.json` で環境変数を使う場合：

```
{
  "mcpServers": {
    "slack": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-slack"],
      "env": {
        "SLACK_BOT_TOKEN": "${SLACK_BOT_TOKEN}"
      }
    }
  }
}
```

`.mcp.json` では `${VAR}` 形式の環境変数展開が使えます。API キーは `.mcp.json` に直書きせず、シェルの環境変数から参照するのが安全です。

5.2.5 SSE (Server-Sent Events) トランスポートについて

注意: SSE トランスポートは**非推奨**です。新規設定には HTTP トランスポートを使ってください。

後方互換のため既存の SSE サーバーは引き続き動作しますが、新しいリモートサーバーの接続には Streamable HTTP を使用してください：

```
# 非推奨 (SSE)
```

```
claude mcp add --transport sse asana https://mcp.asana.com/sse
```

```
# 推奨 (HTTP)
```

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

5.2.6 複数サーバーの同時使用 (`.mcp.json`)

```
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
"/Users/me/projects"]
    },
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "Authorization": "Bearer ${GITHUB_PAT}" }
    },
    "sentry": {
      "type": "http",
      "url": "https://mcp.sentry.dev/mcp"
    }
  }
}
```

5.2.7 OAuth 認証 (リモートサーバー)

HTTP サーバーで OAuth 2.0 認証が必要な場合：

```
# サーバーを追加
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp

# OAuth フローを実行（ブラウザが開きます）
claude mcp login sentry

# または Claude Code セッション内で
/mcp
```

5.2.8 MCP サーバーの動作確認

> 利用可能な MCP ツールを一覧表示して

または Claude Code セッション内で：

```
/mcp
```

GitHub MCP サーバーが正しく設定されていれば：

- > my-project の未解決 Issue を優先度順に並べて
- > Issue #42 に「調査中」のラベルを付けて、担当者を自分に設定して

5.2.9 セキュリティの注意

- API キーは `.mcp.json` に直書きせず、環境変数（`${VAR}`）で参照する
- 個人の API キーは `settings.local.json` や シェルの環境変数で管理し、git 管理外にする
- 必要最小限のスコープ・権限を付与する
- 本番 DB への MCP 接続は慎重に行う

- 各サーバーは信頼できることを確認してから接続する（プロンプトインジェクションリスク）

5.2.10 🛠️ 練習問題

1. **【確認】** stdio 型と Streamable HTTP 型の MCP サーバーの違いを説明してください。また、SSE トランスポートの現在の位置づけも答えてください。
2. **【実践】** `@modelcontextprotocol/server-filesystem` を `claude mcp add` コマンドで追加し、`claude mcp list` で確認してみましょう。
3. **【実践】** 設定した MCP サーバーが認識されているか `/mcp` コマンドで確認しましょう。
4. **【応用】** MCP サーバーの API キーを安全に管理するベストプラクティスを説明してください（`.mcp.json` での環境変数展開の活用を含めて）。

5.2.11 次のステップ

→ 5-3: エージェント SDK に進む

5.3 5-3: エージェント SDK

学習時間: 20分 | 難易度: ★★ ★

5.3.1 Claude Agent SDK とは

Claude Agent SDK は、Claude Code のエージェント機能をプログラムから呼び出すための SDK です。カスタムエージェントの構築や、Claude Code の機能を自前のアプリに組み込みます。TypeScript と Python の両方をサポートしています。

5.3.2 SDK のインストール

```
# TypeScript / JavaScript
npm install @anthropic-ai/claude-agent-sdk

# Python (Python 3.10 以上が必要)
pip install claude-agent-sdk
```

注意: パッケージ名が変更されました。旧パッケージ（`@anthropic-ai/claude-code`）ではなく、`@anthropic-ai/claude-agent-sdk` を使用してください。

5.3.3 基本的な使い方

5.3.3.1 シンプルなエージェント実行

```
// TypeScript
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "src/utils/helpers.ts を読んで、改善点を教えて",
  options: {
    maxTurns: 5,
  },
})) {
  if ("result" in message) console.log(message.result);
}
```

```
# Python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="src/utils/helpers.py を読んで、改善点を教えて",
        options=ClaudeAgentOptions(max_turns=5),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

5.3.3.2 ツールの制限

セキュリティのため、使用するツールを制限できます：

```
// TypeScript
for await (const message of query({
  prompt: "プロジェクトのファイル構造を分析して",
  options: {
    allowedTools: ["Read", "Glob", "Grep"],
    maxTurns: 10,
  },
})) {
  if ("result" in message) console.log(message.result);
}
```

```
# Python
async for message in query(
    prompt="プロジェクトのファイル構造を分析して",
    options=ClaudeAgentOptions(
        allowed_tools=["Read", "Glob", "Grep"],
        max_turns=10,
    ),
):
    if hasattr(message, "result"):
        print(message.result)
```

5.3.3.3 ストリーミング

結果をリアルタイムでストリーミングできます：

```
// TypeScript
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
    prompt: "大きなリファクタリングをして",
    options: { maxTurns: 20 },
})) {
    console.log(message);
}
```

5.3.4 メッセージタイプ

SDK が返すメッセージの種類：

| タイプ | 内容 |
|-----------|---|
| system | セッション情報（ subtype: "init" でセッション ID を取得可能） |
| assistant | Claude のレスポンス |
| user | ツール実行結果など |
| result | 最終結果（成功/失敗） |

5.3.5 オプション一覧

```
// TypeScript
interface QueryOptions {
  maxTurns?: number;           // 最大ターン数
  allowedTools?: string[];      // 許可するツール
  disallowedTools?: string[];   // 禁止するツール
  systemPrompt?: string;        // システムプロンプトの追加
  cwd?: string;                 // 作業ディレクトリ
  model?: string;               // 使用モデル
  permissionMode?: string;      // 権限モード ("acceptEdits" 等)
  mcpServers?: object;          // MCP サーバー設定
  agents?: object;              // カスタムエージェント定義
  resume?: string;              // セッション再開 (セッション ID)
}
```

```
# Python (ClaudeAgentOptions の主なフィールド)
ClaudeAgentOptions(
    max_turns=10,
    allowed_tools=["Read", "Edit"],
    disallowed_tools=[],
    system_prompt="...",
    cwd="/path/to/project",
    model="...",
    permission_mode="acceptEdits",
    mcp_servers={},
    agents={},
    resume="session-id",
)
```

5.3.6 セッション管理

セッション ID を保持することで、会話の文脈を引き継いで続けられます：

```
// TypeScript
let sessionId: string | undefined;

// 最初のクエリでセッション ID を取得
for await (const message of query({ prompt: "認証モジュールを読んで" })) {
  if (message.type === "system" && message.subtype === "init") {
    sessionId = message.session_id;
  }
}

// セッションを再開して続きを実行
for await (const message of query({
  prompt: "それを呼び出している箇所を全部見つけて", // 「それ」=認証モジュール
  options: { resume: sessionId },
})) {
  if ("result" in message) console.log(message.result);
}
```

5.3.7 実践例: 自動コードレビュースクリプト

```
import { query } from "@anthropic-ai/claude-agent-sdk";
import { execSync } from "child_process";

async function reviewPR() {
  const diff = execSync(`git diff main...HEAD`).toString();

  for await (const message of query({
    prompt: `以下の diff をレビューして、問題点を JSON 形式で返して：
\n\n${diff}`,
    options: {
      allowedTools: ["Read", "Glob", "Grep"],
      maxTurns: 5,
    },
  })) {
    if ("result" in message) {
      console.log(JSON.parse(message.result));
    }
  }
}
```

5.3.8 実践例: CI/CD での使用

```
# .github/workflows/review.yml
- name: AI Code Review
  run: |
    node scripts/ai-review.js
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

```
// scripts/ai-review.js
const { query } = require("@anthropic-ai/claude-agent-sdk");

async function main() {
  for await (const message of query({
    prompt: "このブランチの変更をセキュリティの観点でレビューして",
    options: { maxTurns: 10 },
  })) {
    if ("result" in message && message.result.includes("HIGH_SEVE
RITY")) {
      process.exit(1);
    }
  }
}

main().catch(console.error);
```

5.3.9 🛠️ 練習問題

1. **【確認】** Claude Agent SDK でエージェントを起動する際に最低限必要なパラメータを教えてください。また、TypeScript と Python でパッケージ名がどう違うか確認してください。
2. **【実践】** SDK を使って、指定ファイルの要約を出力する簡単なスクリプトを書いてみましょう（`allowedTools: ["Read"]` に限定して）。
3. **【実践】** `maxTurns` を 3 に設定したエージェントを実行し、ターン数制限がどのように機能するか確認しましょう。
4. **【応用】** SDK でエージェントを使う場合と CLI を直接使う場合の使い分けを考えてみましょう（CI/CD・自動化・対話的作業の観点から）。

5.3.10 次のステップ

→ 5-4: マルチエージェント に進む

5.4 5-4: マルチエージェント

学習時間: 20分 | 難易度: ★★ ★

5.4.1 マルチエージェントとは

複数の Claude エージェントを並列または順次に行き、複雑なタスクを効率的に処理する手法です。Claude Code はメインエージェント（オーケストレーター）からサブエージェントを起動できます。

5.4.2 なぜマルチエージェントが必要か

| 問題 | マルチエージェントによる解決 |
|------------------------|----------------------|
| コンテキスト
ウィンドウの
上限 | タスクを分割して各エージェントに割り当て |
| 複数の独立した調査 | 並列実行で高速化 |
| 専門的なレビュー | 役割特化したエージェントを使い分け |
| 大規模コードベース | 並列に異なる部分を分析 |

5.4.3 2つの利用方法

マルチエージェントには **用途が異なる2つの呼び出し方**があります：

| 方法 | 誰が呼び出すか | コードは必要か | 用途 |
|-----------------------|--------------|-----------------------|--------------------------------|
| チャット
（自
動） | Claude が自動判断 | 不要 | Claude
Code を
使う日常
作業 |
| Agent S
DK（直
接） | 自分のスクリプト | 必要（TypeScript/Python） | 独自アプリに組み込む場合 |

5.4.4 Claude Code での使い方（チャット・コード不要）

チャットで指示するだけで、Claude が自動的にサブエージェントを起動します。ユーザー側でコードを書く必要はありません。

- > src/components/ 以下の全コンポーネントをセキュリティレビューして。
- > 並列で複数のエージェントを使って高速化して

Claude Code は指示を解釈し、内部で Agent ツールを呼び出して並列実行します。

5.4.5 Agent SDK でのマルチエージェント（独自アプリに組み込む場合）

前提: 独自のスクリプトやアプリから Claude Code を制御したい場合に使います。日常的なチャット作業には不要です。

```
// TypeScript
import { query } from "@anthropic-ai/claude-agent-sdk";

async function parallelReview(files: string[]) {
  // 複数ファイルを並列でレビュー
  const reviews = await Promise.all(
    files.map((file) => {
      const messages: string[] = [];
      return (async () => {
        for await (const message of query({
          prompt: `${file} をセキュリティレビューして JSON で返して
          ` ,
          options: {
            allowedTools: ["Read"],
            maxTurns: 3,
          },
        })) {
          if ("result" in message) messages.push(message.result);
        }
        return messages[messages.length - 1];
      })();
    })
  );

  return reviews.map((r) => JSON.parse(r ?? "{}"));
}

// 使用例：3ファイルを並列レビュー
const results = await parallelReview([
  "src/auth/login.ts",
  "src/auth/session.ts",
  "src/api/users.ts",
]);
```

```
# Python
import asyncio
import json
from claude_agent_sdk import query, ClaudeAgentOptions

async def review_file(file: str) -> str:
    result = ""
    async for message in query(
        prompt=f"{file} をセキュリティレビューして JSON で返して",
        options=ClaudeAgentOptions(allowed_tools=["Read"], max_turns=3),
    ):
        if hasattr(message, "result"):
            result = message.result
    return result

async def parallel_review(files: list[str]):
    results = await asyncio.gather(*[review_file(f) for f in files])
    return [json.loads(r) for r in results]
```

5.4.6 オーケストレーターとサブエージェントのパターン

```
// TypeScript: オーケストレーター
async function orchestrate() {
  // Step 1: 調査エージェント（並列）
  const collectResults = async (prompt: string) => {
    let result = "";
    for await (const msg of query({ prompt, options: { maxTurns:
5 } }))) {
      if ("result" in msg) result = msg.result;
    }
    return result;
  };

  const [authIssues, apiIssues, dbIssues] = await Promise.all([
    collectResults("認証コードの問題を調査して"),
    collectResults("API エンドポイントの問題を調査して"),
    collectResults("DB クエリの問題を調査して"),
  ]);

  // Step 2: 修正エージェント（調査結果を基に）
  const fixPrompt = `
  以下の問題を修正して：
  - 認証: ${authIssues}
  - API: ${apiIssues}
  - DB: ${dbIssues}
  `;

  for await (const msg of query({ prompt: fixPrompt, options: { m
axTurns: 20 } }))) {
    if ("result" in msg) console.log(msg.result);
  }
}
```

5.4.7 カスタムサブエージェントの定義

Agent SDK では、`agents` オプションで専門エージェントを定義して呼び出せます：

```
// TypeScript
for await (const message of query({
  prompt: "code-reviewer エージェントを使ってコードをレビューして",
  options: {
    allowedTools: ["Read", "Glob", "Grep", "Agent"],
    agents: {
      "code-reviewer": {
        description: "コード品質とセキュリティのエキスパートレビューア",
        prompt: "コード品質を分析し、改善点を提案する専門エージェントです。",
        tools: ["Read", "Glob", "Grep"],
      },
    },
  },
})) {
  if ("result" in message) console.log(message.result);
}
```

サブエージェントを呼び出すには `allowedTools` に `"Agent"` を含める必要があります。

5.4.8 ワークツリーによる並列開発

Git ワークツリーを使うと、同じリポジトリの別のブランチで複数のエージェントが並列作業できます：

- ＞ 2つのエージェントを使って、
- ＞ エージェント1: パフォーマンス改善
- ＞ エージェント2: セキュリティ修正
- ＞ を並列で行って、最後にマージして

5.4.9 並列実行 vs 直列実行の使い分け

| 実行方法 | 適切な場面 |
|------|--------------------------------|
| 並列実行 | 独立したタスク（互いの結果に依存しない）、高速化が必要な場合 |
| 直列実行 | 前の結果を次のステップで使う場合、順序が重要な場合 |

5.4.10 注意事項

- サブエージェントはそれぞれコストが発生する
- 並列実行はコンテキストを共有しない（各エージェントは独立した会話）
- 結果の統合はオーケストレーターが責任を持つ
- エラー伝播に注意する（1つの失敗が全体に影響しないよう設計する）
- コスト超過に注意し、`maxTurns` で上限を設定する

5.4.11 🧑🎓 練習問題

1. **【確認】** オーケストレーターとサブエージェントの役割の違いを説明してください。
2. **【確認】** サブエージェントを並列実行する場合と直列実行する場合の使い分けを教えてください。
3. **【実践】** 2つの独立したタスク（例: 「src/auth/ のセキュリティ問題を調査」と「src/api/ のパフォーマンス問題を調査」）を並列サブエージェントで実行する

スクリプトを作成してみましょう。

4. **【応用】** マルチエージェントシステムを設計する際のリスク（コスト・エラー伝播・コンテキスト非共有等）を2つ挙げ、それぞれの対策を説明してください。

5.4.12 次のステップ

→ 5-5: カスタムエージェント実装 に進む

5.5 5-5: カスタムエージェント実装

学習時間: 30分 | 難易度: ★★ ★

5.5.1 カスタムエージェントとは

特定のタスクに特化したエージェントを定義し、再利用できます。Agent SDK では `agents` オプションで直接定義する方法と、Claude Code のスキル（`.claude/skills/` ディレクトリ）を利用する方法があります。

5.5.2 カスタムエージェントの設計原則

1. **単一責務**: 1つのエージェントは1つのことをうまくやる
2. **ツール制限**: 必要最小限のツールだけ許可する
3. **明確なインターフェース**: 入力と出力の形式を明確にする
4. **エラーハンドリング**: 失敗ケースを想定した設計

5.5.3 Agent SDK でのカスタムエージェント定義

5.5.3.1 TypeScript

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "code-reviewer エージェントを使ってこのコードをレビューして",
  options: {
    allowedTools: ["Read", "Glob", "Grep", "Agent"],
    agents: {
      "code-reviewer": {
        description: "コード品質とセキュリティのエキスパートレビューア。コードレビューを依頼されたときに使う。",
        prompt: "コード品質を分析し、バグ・セキュリティ問題・改善点を報告する専門エージェントです。",
        tools: ["Read", "Glob", "Grep"],
      },
      "doc-generator": {
        description: "JSDoc / docstring を自動生成する専門エージェント。",
        prompt: "コードを読み取り、適切なドキュメントコメントを生成します。",
        tools: ["Read"],
      },
    },
  },
})) {
  if ("result" in message) console.log(message.result);
}
```

5.5.3.2 Python

```
import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

async def main():
    async for message in query(
        prompt="code-reviewer エージェントを使ってこのコードをレビューして",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep", "Agent"],
            agents={
                "code-reviewer": AgentDefinition(
                    description="コード品質とセキュリティのエキスパートレビューアー。",
                    prompt="コード品質を分析し、バグ・セキュリティ問題・改善点を報告します。",
                    tools=["Read", "Glob", "Grep"],
                ),
                "doc-generator": AgentDefinition(
                    description="docstring を自動生成する専門エージェント。",
                    prompt="コードを読み取り、適切なドキュメントコメントを生成します。",
                    tools=["Read"],
                ),
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)
```

```
asyncio.run(main())
```

サブエージェントを呼び出すには `allowedTools` (Python: `allowed_tools`) に `"Agent"` を含める必要があります。

5.5.4 実装例: PR レビューエージェント

```
import { query } from "@anthropic-ai/claude-agent-sdk";

interface PRReviewResult {
  score: number;
  issues: Array<{
    severity: "critical" | "major" | "minor";
    file: string;
    line: number;
    message: string;
  }>;
  approved: boolean;
  summary: string;
}

async function prReviewAgent(prDiff: string): Promise<PRReviewResult> {
  let resultText = "";

  for await (const message of query({
    prompt: `
以下の PR diff を厳密にレビューして、JSON で結果を返して。

## レビュー観点
1. バグ・ロジックエラー (critical)
2. セキュリティ問題 (critical)
3. パフォーマンス問題 (major)
4. コード品質 (minor)

## 出力形式 (必ずこの JSON のみを返すこと)
{
  "score": 0-100,
```

```

    "issues": [
      {
        "severity": "critical|major|minor",
        "file": "ファイルパス",
        "line": 行番号,
        "message": "問題の説明"
      }
    ],
    "approved": true/false,
    "summary": "1～2文のサマリー"
  }

## Diff
${prDiff}
`,
  options: {
    allowedTools: ["Read", "Grep"],
    maxTurns: 5,
  },
})) {
  if ("result" in message) resultText = message.result;
}

const jsonMatch = resultText.match(/{\[\\s\\S]*\\}/);
if (!jsonMatch) throw new Error("Invalid response format");

return JSON.parse(jsonMatch[0]) as PRReviewResult;
}

```

5.5.5 実装例: ドキュメント生成エージェント

```
async function docGeneratorAgent(filePath: string): Promise<string> {
```

```
    let resultText = "";
```

```
    for await (const message of query({
        prompt: `
```

`${filePath}` を読んで、以下の形式で JSDoc コメントを生成して。
コメントのみを返し、コード自体は変更しないこと。

形式:

```
/**
```

```
 * 関数の説明
```

```
 * @param {型} 引数名 - 説明
```

```
 * @returns {型} 返り値の説明
```

```
 * @example
```

```
 * // 使用例
```

```
 */
```

```
`,
```

```
options: {
```

```
    allowedTools: ["Read"],
```

```
    maxTurns: 3,
```

```
},
```

```
})) {
```

```
    if ("result" in message) resultText = message.result;
```

```
}
```

```
    return resultText;
```

```
}
```

5.5.6 エージェントのパイプライン化

複数のエージェントをパイプラインとして繋がります：

```
async function qualityPipeline(files: string[]) {
  for (const file of files) {
    // Step 1: レビュー
    const review = await prReviewAgent(await getFileDiff(file));

    if (review.issues.some((i) => i.severity === "critical")) {
      // Step 2: 自動修正 (critical のみ)
      for await (const msg of query({
        prompt: `${file} の以下の問題を修正して : ${JSON.stringify(r
review.issues.filter((i) => i.severity === "critical"))}`,
        options: { maxTurns: 10 },
      }))) {
        // 進捗をログ
      }

      // Step 3: 修正後に再レビュー
      const reReview = await prReviewAgent(await getFileDiff(fil
e));
      console.log(`${file}: ${reReview.approved ? "OK" : "NG"}`);
    }
  }
}
```

5.5.7 専門エージェント vs 汎用エージェントの比較

| 観点 | 専門エージェント（複数） | 汎用エージェント（単一） |
|------|----------------|---------------|
| 精度 | 高い（タスクに特化） | 中程度 |
| コスト | 高い（複数回の呼び出し） | 低い（1回で完結） |
| 再利用性 | 高い（複数箇所で使い回せる） | 低い |
| 保守性 | 明確な責務分担で管理しやすい | シンプルで管理が楽 |
| 適用場面 | 複雑・大規模なタスク | シンプルな一回限りのタスク |

5.5.8 GitHub Actions でのカスタムエージェント

```
name: AI Quality Gate
on: [pull_request]

jobs:
  ai-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AI Review Agent
        run: node scripts/pr-review-agent.js
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
          PR_DIFF: ${ github.event.pull_request.diff_url }
```

5.5.9 🧑‍🎓 練習問題

1. **【確認】** Agent SDK でカスタムエージェントを定義する際に必要なフィールド（`description`・`prompt`・`tools`）の役割をそれぞれ説明してください。
2. **【実践】** 利用可能なツールを `Read` と `Grep` のみに限定した「コードリーダー」エージェントを Agent SDK で定義し、`Agent` ツール経由で呼び出すスクリプトを書いてみましょう。
3. **【実践】** 定義したカスタムエージェントを実際に呼び出し、結果を確認してみましょう。
4. **【応用】** 専門エージェントを複数用意するメリットと、単一の汎用エージェントを使うメリットを比較し、どのような場合にどちらを選ぶべきか説明してください。

5.5.10 次のステップ

→ 6-1: CI/CD 統合 に進む

6. 発展・応用

6.1 6-1: CI/CD 統合

学習時間: 20分 | 難易度: ★★ ★

6.1.1 概要

Claude Code を CI/CD パイプラインに組み込むことで、コードレビューや品質チェックを自動化できます。公式の `claude-code-action` を使うと、GitHub PR やイシューへの `@claude` メンションをトリガーにして AI による自動実装・レビューが行えます。

6.1.2 GitHub Actions での基本的な使い方

6.1.2.1 クイックセットアップ（推奨）

Claude Code のターミナルで以下を実行すると、GitHub App のインストールからワークフロー設定まで対話式に完結します：

```
/install-github-app
```

注意: リポジトリの管理者権限が必要です。Amazon Bedrock や Google Vertex AI を使用する場合は後述の手動セットアップが必要です。

6.1.2.2 手動セットアップ

1. Claude GitHub App をインストール: <https://github.com/apps/claude>
 - 必要な権限: Contents・Issues・Pull requests (Read & Write)
2. `ANTHROPIC_API_KEY` をリポジトリシークレットに追加
3. ワークフローファイルをコピー: `examples/claude.yml`

6.1.2.3 基本ワークフロー（@claude メンション対応）

```
# .github/workflows/claude.yml
name: Claude Code

on:
  issue_comment:
    types: [created]
  pull_request_review_comment:
    types: [created]
  issues:
    types: [opened, assigned]

jobs:
  claude:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
      issues: write
    steps:
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          # PR/イシューコメントで @claude を呼ぶと自動で応答する
```

コメントで @claude をメンションするだけで Claude が動作します：

```
@claude この機能を実装して
@claude このバグを修正して
@claude セキュリティレビューをお願い
```

6.1.2.4 PR 自動コードレビュー（スケジュール実行）

PR が開かれたタイミングで自動的にレビューを実行する場合は `prompt` パラメータを使います：

```
# .github/workflows/ai-review.yml
name: AI Code Review

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          prompt: "この PR をレビューして。バグ・セキュリティ・パフォーマンスの問題を報告して"
          claude_args: "--max-turns 5"
```

6.1.2.5 Claude Agent SDK を使ったカスタム統合

より高度なカスタム処理が必要な場合は、`@anthropic-ai/claude-agent-sdk` を使ってスクリプトを書きます：

```
// scripts/review.js
const { query } = require("@anthropic-ai/claude-agent-sdk");
const { Octokit } = require("@octokit/rest");
const fs = require("fs");

async function main() {
  const diff = fs.readFileSync("/tmp/pr.diff", "utf8");

  const result = await query({
    prompt: `
以下の PR diff をレビューして。
重大な問題（バグ・セキュリティ・パフォーマンス）のみを報告して。
JSON 形式で返して。

${diff}
`,
    options: {
      allowedTools: ["Read", "Grep"],
      maxTurns: 5,
    },
  });

  const issues = JSON.parse(result.result);

  if (issues.critical_count > 0) {
    console.error("Critical issues found");
    process.exit(1);
  }

  console.log("Review passed");
}

main().catch(console.error);
```

6.1.3 GitLab CI での使用

6.1.3.1 GitLab CI の基本概念

GitLab CI は `.gitlab-ci.yml` という1つのファイルでパイプライン全体を定義します。

| 用語 | 意味 | GitHub Actions との
対応 |
|--------------------|--|-------------------------|
| Pipeline | CI 全体の実行単位 | Workflow |
| Stage | パイプラインの段階（例:
build → test → review） | - |
| Job | 各ステージ内の個別タスク | Job |
| Runner | ジョブを実行するサーバー | Runner |
| Merge Request (MR) | コードレビューの単位 | Pull Request (PR) |

6.1.3.2 API キーの設定方法

GitLab プロジェクトの **Settings** → **CI/CD** → **Variables** で以下を登録します：

| 変数名 | 値 | オプション |
|--------------------------------|-------------------------|------------------|
| <code>ANTHROPIC_API_KEY</code> | <code>sk-ant-...</code> | Masked（ログに表示しない） |

6.1.3.3 基本設定

```
# .gitlab-ci.yml
ai-review:
  stage: review
  image: node:20
  script:
    - npm install -g @anthropic-ai/claude-agent-sdk
    - git diff origin/main...HEAD > /tmp/pr.diff
    - node scripts/review.js
  variables:
    ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY
  only:
    - merge_requests
```

6.1.3.4 設定の各項目の意味

| キー | 説明 |
|----------------------|-------------------------------------|
| stage: review | stages: で定義したステージの1つに配置する |
| image: node:20 | ジョブを実行する Docker イメージ（Node.js 20） |
| script: | 実行するシェルコマンドのリスト（上から順に実行） |
| variables: | ジョブ内で使う環境変数（\$変数名で GitLab CI 変数を参照） |
| only: merge_requests | マージリクエストが作成・更新されたときだけ実行 |

6.1.3.5 複数ステージを使った実践的な構成

```
# .gitlab-ci.yml (マルチステージ版)

stages:
  - build
  - test
  - review    # ← AI レビューはここ
  - deploy

ai-review:
  stage: review
  image: node:20
  script:
    - npm install -g @anthropic-ai/claude-agent-sdk
    - git diff origin/main...HEAD > /tmp/pr.diff
    - node scripts/review.js
  variables:
    ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY
  artifacts:
    paths:
      - review-report.json    # レポートを次のジョブに引き継ぐ
    expire_in: 1 week
  only:
    - merge_requests
```

ポイント: `artifacts` を使うとレビュー結果ファイルを保存・ダウンロードできます。

6.1.3.6 GitLab と GitHub Actions の比較

| 項目 | GitLab CI | GitHub Actions |
|----------------|---|--|
| 設定ファイル | <code>.gitlab-ci.yml</code> (1
ファイル) | <code>.github/workflows/*.yml</code> (複
数可) |
| トリガー | <code>only: merge_reques
ts</code> | <code>on: pull_request:</code> |
| シークレット | Settings → CI/CD →
Variables | Settings → Secrets |
| セルフホスト
実行環境 | GitLab Runner | Self-hosted Runner |

6.1.4 品質ゲートとしての使用

コードレビューが通らなければマージをブロックするパターン：

```
# GitHub Actions
- name: AI Security Gate
  uses: anthropics/claude-code-action@v1
  with:
    anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
    prompt: "このブランチの変更をセキュリティレビューして。HIGH 以上の問題があれば最後に SECURITY_GATE_FAILED と出力して"
    claude_args: "--max-turns 5"
```

6.1.5 コスト管理

CI/CD で使用する場合はコストに注意：

| 戦略 | 説明 |
|------------------------------|--|
| <code>--max-turns</code> を制限 | <code>claude_args: "--max-turns 5"</code>
でターン数を制限 |
| 軽量モデルを使う | <code>claude_args: "--model claude-haiku-4-5"</code> を CI/CD に |
| 変更ファイルのみ対象 | diff ファイルのみを読む |
| ワークフロータイムアウト設定 | 暴走ジョブを防ぐために <code>timeout-minutes</code> を設定 |
| 並列実行数を制限 | GitHub の concurrency 設定でコスト上限を管理 |

6.1.6 Amazon Bedrock / Google Vertex AI での利用

企業環境では、`ANTHROPIC_API_KEY` の代わりに自社クラウドを使用できます：

```
# Amazon Bedrock の場合
- uses: anthropics/claude-code-action@v1
  with:
    use_bedrock: "true"
    claude_args: '--model us.anthropic.claude-sonnet-4-6 --max-turns 10'
    # AWS 認証は configure-aws-credentials@v4 アクションで事前設定

# Google Vertex AI の場合
- uses: anthropics/claude-code-action@v1
  with:
    use_vertex: "true"
    claude_args: '--model claude-sonnet-4-5@20250929 --max-turns 10'
  env:
    ANTHROPIC_VERTEX_PROJECT_ID: ${ steps.auth.outputs.project_id }
    CLOUD_ML_REGION: us-east5
```

6.1.7 🧑🎓 練習問題

1. **【確認】** GitHub Actions で Claude Code を使う際、リポジトリシークレットに設定が必要な環境変数名を教えてください。また、`/install-github-app` コマンドで何ができるか説明してください。
2. **【実践】** `claude-code-action@v1` を使った GitHub Actions ワークフロー（PR 作成時に自動でコードレビューを実行）を作成してみましょう。`prompt` パラメーターと `claude_args` の両方を使ってください。
3. **【実践】** `@anthropic-ai/claude-agent-sdk` の `query` 関数を使った CI スクリプトを書いてみましょう。`allowedTools` と `maxTurns` を設定して、diff をレビューするサンプルを実装してください。

4. 【応用】 Claude Code を CI に組み込む際のコスト管理のポイントを2つ挙げてください。 `claude_args` でどのように制御できるかも合わせて説明してください。

6.1.8 次のステップ

→ 6-2: チームワークフロー に進む

6.2 6-2: チームワークフロー

学習時間: 15分 | 難易度: ★ ★ ★

6.2.1 チームでの Claude Code 活用

チームで Claude Code を使う際は、個人の設定とチーム共有の設定を適切に分離することが重要です。

6.2.2 設定の分離

```
my-project/
├── .claude/
│   ├── settings.json          # チームで共有 (git 管理)
│   ├── settings.local.json    # 個人設定 (.gitignore に追加)
│   └── skills/                 # チーム共有スキル (git 管理)
│       ├── deploy-check/
│       └── release-notes/
├── CLAUDE.md                  # プロジェクト指示 (git 管理)
└── .gitignore
    # .claude/settings.local.json を追加
```

| ファイル | 共有範囲 | 内容の例 |
|--|------------------|---------------------|
| <code>.claude/settings.json</code> | チーム全体（git 管理） | 許可コマンド・フック・チーム共通ルール |
| <code>.claude/settings.local.json</code> | 個人のみ（.gitignore） | 個人の API キー・個人の好み |
| <code>CLAUDE.md</code> | チーム全体（git 管理） | プロジェクト概要・コーディング規約 |

6.2.2.1 .gitignore に追加

```
# Claude Code 個人設定
.claude/settings.local.json
```

6.2.3 チーム共有の settings.json

```
{
  "permissions": {
    "allow": [
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(npm run *)",
      "Bash(npm test)"
    ],
    "deny": [
      "Bash(git push --force *)",
      "Bash(rm -rf *)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npm run format 2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

6.2.4 効果的な CLAUDE.md の書き方（チーム向け）

プロジェクト名

チームルール

- PR は必ず 1 機能ずつに分割する
- マージ前に `/code-review` を実行すること
- ブランチ名は `feature/`, `fix/`, `chore/` プレフィックスを使う

禁止事項

- main ブランチへの直接 push
- console.log のコミット
- `any` 型の使用

コードオーナー

- src/auth/ → @auth-team
- src/payments/ → @payments-team

Compact instructions

When you are using compact, please focus on test output and code changes

ポイント: CLAUDE.md は 200 行以内に収めましょう。詳細な手順は SKILL.md に切り出すとコンテキスト節約になります。

6.2.5 チーム共有スキルの管理

プロジェクトスキルをリポジトリに含めることでチーム全員が同じスキルを使えます：

```
.claude/skills/
├─ release-notes/
│   └─ SKILL.md    # リリースノート自動生成
├─ db-migration/
│   └─ SKILL.md    # DB マイグレーション安全チェック
└─ api-review/
    └─ SKILL.md    # API 設計レビュー
```

スキルはオンデマンドで読み込まれるため、CLAUDE.md に詳細手順を書くよりコンテキストを節約できます。

6.2.6 オンボーディングへの活用

新メンバーのオンボーディングに CLAUDE.md を活用：

```
## 新メンバーへ
```

このプロジェクトに初めて参加する場合は、以下を実行してください：

```
```bash
npm install
cp .env.example .env.local
npm run db:setup
npm run dev
```
```

Claude Code で ``/init`` を実行すると、プロジェクト全体の概要を説明します。

6.2.7 レビュー文化への組み込み

6.2.7.1 PR テンプレートに Claude Code を組み込む

```
<!-- .github/pull_request_template.md -->
## チェックリスト

- [ ] `/code-review` を実行した
- [ ] `/security-review` を実行した（認証・DB 変更がある場合）
- [ ] テストを追加・更新した
- [ ] CLAUDE.md のコーディング規約を確認した
```

6.2.8 チームコストの管理

チームで使用する場合は、Anthropic Console でコストを管理できます：

- **ワークスペース支出上限:** Console の Workspace 設定でチーム全体の上限を設定
- **使用量レポート:** Console の Usage ページでチームの API 使用量を確認
- **レート制限の推奨値**（ユーザー数に応じた目安）：

| チームサイズ | TPM/ユーザー | RPM/ユーザー |
|---------|-----------|-----------|
| 1～5名 | 200k～300k | 5～7 |
| 5～20名 | 100k～150k | 2.5～3.5 |
| 20～50名 | 50k～75k | 1.25～1.75 |
| 50～100名 | 25k～35k | 0.62～0.87 |

6.2.9 🏆 練習問題

1. **【確認】** チームで共有すべき設定ファイルと個人設定に留めるべきファイルをそれぞれ答えてください。なぜそのように分けるのかも説明してください。
2. **【実践】** `.claude/settings.json` にチーム共通の `allowedTools` (git コマンドと npm スクリプト) の許可設定を追加し、`deny` リストに危険なコマンドを含めて、git にコミットしてみましょう。
3. **【実践】** CLAUDE.md にチームのコーディング規約 (ブランチ命名規則・コミットメッセージ規則・コードレビュールール) を追記し、Claude Code がそれを遵守するか確認しましょう。
4. **【応用】** 新しいチームメンバーが Claude Code をセットアップする際の「オンボーディングチェックリスト」を作成してみましょう。インストール方法・API キー設定・CLAUDE.md の読み方・最初に試すべきコマンドを含めてください。

6.2.10 次のステップ

→ 6-3: パフォーマンス Tips に進む

6.3 6-3: パフォーマンス Tips

学習時間: 15分 | 難易度: ★★ ★

6.3.1 レスポンス速度の最適化

6.3.1.1 用途に合ったモデルを選ぶ

`/model` コマンドでセッション中にモデルを切り替えられます。デフォルトは `/config` で設定できます。

| タスク | 推奨モデル |
|--------------------|---|
| 複雑なアーキテクチャ設計・多段階推論 | <code>claude-opus-4-6</code> |
| 通常の開発作業（バランス重視） | <code>claude-sonnet-4-6</code> |
| 単純な編集・整形・CI チェック | <code>claude-haiku-4-5</code> |
| サブエージェントの調整タスク | <code>claude-sonnet-4-6</code> （コスト・品質バランス） |

拡張思考（Extended Thinking）について: Opus や Sonnet では深い推論のための拡張思考が使われます。複雑なタスクでは精度が上がりますが、思考トークンも課金対象です。シンプルなタスクでは `/effort` コマンドで思考量を下げるとコスト削減になります。

6.3.2 コンテキストの最適化

6.3.2.1 不要なファイルを読ませない

- # 悪い例
- > プロジェクト全体を読んで、Button コンポーネントを修正して
- # 良い例
- > `src/components/Button.tsx` を読んで修正して

6.3.2.2 CLAUDE.md を簡潔に保つ

CLAUDE.md は毎回読み込まれます。**200 行以内**を目標に、必要最小限の情報だけ書きましょう：

- 技術スタック（5～10行）
- よく使うコマンド（5～10個）
- 重要な規約（5～10行）

詳細なワークフロー手順はスキル（SKILL.md）に切り出すことで、必要なときだけ読み込まれます。

6.3.2.3 /clear と /compact を適切に使う

| コマンド | 用途 |
|-----------------------|---------------------------------------|
| <code>/clear</code> | タスク完了後にコンテキストを完全リセット（関連するスタックした会話を除去） |
| <code>/compact</code> | 会話を要約してコンテキストを圧縮（会話の継続が必要な場合） |

`/compact` にフォーカスを指定するとより効果的です：

`/compact` コードの変更点と残タスクだけ保持して

CLAUDE.md に `compact` の挙動を設定することもできます：

```
# Compact instructions
```

When you are using compact, please focus on test output and code changes

6.3.2.4 /usage でコンテキスト使用量を確認

```
/usage
```

現在のセッションのトークン使用量とコストを確認できます。ステータスラインにコンテキスト使用率を常時表示する設定もあります。

6.3.3 プロンプトの最適化

6.3.3.1 具体的に指示する

```
# 悪い例（曖昧）
```

```
> このコードを改善して
```

```
# 良い例（具体的）
```

```
> UserList コンポーネントの useEffect を最適化して。
```

```
> 現在は毎レンダリングで API 呼び出しが発生している。
```

```
> userId が変わったときだけ呼び出すよう修正して。
```

6.3.3.2 出力形式を指定する

```
> テストカバレッジが低い関数を10個、JSON 配列で返して。
```

```
> 各要素は { file, function, currentCoverage } の形式で。
```

6.3.3.3 複雑なタスクでは Plan モードを活用

Shift+Tab でプランモードに切り替えると、Claude が実装前にアプローチを計画・提示します。誤った方向への大量トークン消費を防げます。

```
# Plan モードに切り替え
Shift+Tab を押す

# Claude が計画を提示 → 承認してから実装
```

6.3.4 API コストの最適化

6.3.4.1 プロンプトキャッシングの活用

長い CLAUDE.md や参照ドキュメントは自動的にキャッシュされます。繰り返し参照するドキュメントは CLAUDE.md に書いておくとコストが下がります。

キャッシュの仕組み： - システムプロンプトやよく使うコンテンツは自動キャッシュ - キャッシュヒット時はトークンコストが大幅削減 - **@include** でファイルを参照するとキャッシュ効率が上がる

6.3.4.2 CI/CD での最適化

```
// CI/CD では軽量モデルを使う
const result = await query({
  prompt: "...",
  options: {
    model: "claude-haiku-4-5", // 高速・低コスト
    maxTurns: 3,                // ターン数を制限
    allowedTools: ["Read", "Grep"], // ツールを制限
  },
});
```

6.3.4.3 フックとスキルでコンテキスト節約

フック: 大量データを Claude が読む前に前処理できます。

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "~/claude/hooks/filter-test-output.sh"
      }]
    }]
  }
}
```

例: テスト実行時に失敗した行だけ抽出して Claude に渡す → 10,000 行が数十行に圧縮。

スキル: 複雑なワークフロー手順をオンデマンドで読み込みます。CLAUDE.md に常駐させる必要がなくなります。

6.3.4.4 MCP サーバーの整理

`/mcp` で設定済みサーバーを確認し、使っていないものは無効化しましょう。MCP ツール定義はデフォルトで遅延読み込みされますが、不要なサーバーは起動オーバーヘッドになります。

6.3.5 サブエージェントの活用

時間のかかる調査はサブエージェントに任せることでメインの会話をブロックせずに進められます：

- ＞ バックグラウンドで `src/legacy/` の依存関係を調査して。
- ＞ 終わったら教えて。それまで別の作業を続けよう。

サブエージェントのモデルを指定してコストを節約：

- ＞ Haiku を使って `src/utils/` 全体のコメントを英語に翻訳して

6.3.6 よくあるボトルネック

| 症状 | 原因 | 対処法 |
|---------------|------------------|--|
| レスポンスが遅い | コンテキストが大きすぎる | <code>/compact</code> または <code>/clear</code> でリセット |
| 同じ情報を何度も確認する | CLAUDE.md が不完全 | CLAUDE.md を充実させる |
| ファイルを何度も読み直す | 関連ファイルを先に読ませる | 最初に必要ファイルを指定 |
| 承認待ちが多い | permissions が未設定 | settings.json を設定 |
| CPU/メモリ使用率が高い | 大規模コードベース処理中 | <code>/compact</code> を定期実行、ビルドディレクトリを <code>.gitignore</code> に |
| 自動圧縮がスラッシング | 1ファイルが巨大すぎる | 行範囲を指定して読む、またはサブエージェントに委譲 |

6.3.7 🏆 練習問題

1. 【確認】 Claude Code でコストを削減するために有効な方法を3つ挙げてください。それぞれが「コンテキスト削減」「モデル選択」「フック・スキル活用」のどのカテゴリに属するかも説明してください。
2. 【実践】 大きなプロジェクトで作業する際に `/compact` と `/clear` をどのタイミングで使い分けるか、実際に長い会話を行いながら意識してみましょう。`/usage` で使用トークン数の変化を確認してください。
3. 【実践】 `/model` コマンドを使って `claude-haiku-4-5` と `claude-sonnet-4-6` を切り替えながら同じタスクを実行し、品質とレスポンス速度の違いを確認してみましょう。
4. 【応用】 CLAUDE.md を活用してコンテキストの重複読み込みを減らす工夫を考えてみましょう。プロジェクト固有の情報を CLAUDE.md に書く場合と、スキル（SKILL.md）に切り出す場合の使い分け基準を説明してください。

6.3.8 次のステップ

→ 6-4: トラブルシューティング に進む

6.4 6-4: トラブルシューティング

学習時間: 15分 | 難易度: ★★ ★

6.4.1 まず試すこと: `/doctor`

問題が発生したらまずこのコマンドを実行します：

```
/doctor
```

または、Claude Code が起動しない場合はシェルから：

```
claude doctor
```

インストール状態・設定ファイルの妥当性・MCP サーバーの接続・コンテキスト使用量を自動チェックします。

6.4.2 問題別 クイックリファレンス

| 症状 | 参照セクション |
|--|---------|
| <code>command not found: claude</code> | 起動しない |
| <code>ANTHROPIC_API_KEY</code> エラー | 認証エラー |
| ツールが承認待ちになり続ける | ツール承認 |
| スキルが自動読み込みされない | スキル |
| MCP サーバーが接続できない | MCP |
| コンテキストが切れてしまう | コンテキスト |
| フックが動作しない | フック |
| レート制限エラー | API エラー |
| Windows 文字化け | Windows |
| WSL での検索問題 | WSL |
| 応答が遅い・ハング | パフォーマンス |

6.4.3 よくある問題と解決策

6.4.3.1 Claude Code が起動しない

症状: `claude` コマンドが見つからない

```
# 確認
which claude      # macOS/Linux
claude --version  # インストール確認
claude doctor     # 自動診断
```

PATH が通っていない場合:

```
# macOS/Linux (Zsh)
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

# macOS/Linux (Bash)
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

```
# Windows PowerShell
$currentPath = [Environment]::GetEnvironmentVariable('PATH', 'User')
[Environment]::SetEnvironmentVariable('PATH', "$currentPath;$env:
USERPROFILE\.local\bin", 'User')
# ターミナルを再起動
```

再インストール（ネイティブインストーラーが推奨）:

```
# macOS/Linux
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Homebrew (macOS)
brew install --cask claude-code

# WinGet (Windows)
winget install Anthropic.ClaudeCode
```

インストール先: `~/local/bin/claude` (macOS/Linux) /
`%USERPROFILE%\local\bin\claude.exe` (Windows)

6.4.3.2 認証エラー

症状: `ANTHROPIC_API_KEY` が未設定、または `403 Forbidden`

```
# API キーを確認
echo $ANTHROPIC_API_KEY

# 設定
export ANTHROPIC_API_KEY="sk-ant-..."

# 永続化 ( ~/.bashrc または ~/.zshrc に追加)
echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.bashrc
```

症状: `403 Forbidden` でログイン済みなのに動かない

原因の可能性: 1. `ANTHROPIC_API_KEY` 環境変数が残っており、OAuth を上書きしている - `unset ANTHROPIC_API_KEY` で一時的に解除してから `claude` を起動 - `~/ .bashrc` や `~/ .zshrc` から `export ANTHROPIC_API_KEY=...` を削除 2. Claude Pro/Max サブスクリプションの有効期限切れ 3. Anthropic Console でのロール設定不足 (“Claude Code” または “Developer” ロールが必要)

症状: OAuth エラー (WSL2・SSH・コンテナ環境)

ブラウザが別のホストで開き、リダイレクトが Claude Code に届かない場合: - ターミナルに表示されるログインコードを `Paste code here if prompted` プロンプトに貼り付ける - `c` を押して OAuth URL をコピーし、ローカルのブラウザで開く

6.4.3.3 ツールが承認待ちになり続ける

症状: 毎回 `Bash(git status)` の承認を求められる

解決策: `settings.json` に許可ルールを追加する

```
{
  "permissions": {
    "allow": [
      "Bash(git *)",
      "Bash(npm *)",
      "Read(**)",
      "Edit(**)"
    ]
  }
}
```

6.4.3.4 スキルが自動読み込みされない

症状: `.claude/skills/my-skill/SKILL.md` を置いたが自動起動しない

確認ポイント: 1. `description` フィールドが具体的か確認する 2. ディレクトリ名とファイル名が正しいか確認する

```
# 構造を確認
ls .claude/skills/my-skill/
# → SKILL.md が存在すること

# description を確認
head -10 .claude/skills/my-skill/SKILL.md
# → description フィールドが具体的に書かれていること
```

スラッシュコマンドで明示的に呼び出すこともできます：

```
/my-skill
```

6.4.3.5 MCP サーバーが接続できない

症状: MCP ツールが使えない

```
# MCP サーバーを手動で起動してテスト
npx -y @modelcontextprotocol/server-filesystem /tmp

# settings.json の記述を確認
cat ~/.claude/settings.json | grep -A 10 mcpServers
```

よくある原因: - `npx` のパスが通っていない - 環境変数（API キー等）が未設定 - Node.js のバージョンが古い（npm 経由インストールの場合は 18 以上が必要）

検索が機能しない場合（ファイル検索・`@file` が動かない）：

バンドルされた `ripgrep` が動作していない可能性があります。プラットフォーム固有のものをインストールして Claude Code に使わせます：

```
# macOS
brew install ripgrep

# Ubuntu/Debian
sudo apt install ripgrep

# Windows
winget install BurntSushi.ripgrep.MSVC
```

その後、`settings.json` に追加：

```
{
  "env": {
    "USE_BUILTIN_RIPGREP": "0"
  }
}
```

6.4.3.6 コンテキストが切れてしまう

症状: 途中で会話がリセットされる、過去の指示を忘れる

解決策: 1. 重要な指示は CLAUDE.md に書く 2. メモリに保存する（> **これを覚えておいて**） 3. **/compact** でコンテキストを圧縮しながら会話を継続する 4. 長いタスクは小さく分割して **/clear** を挟む

自動圧縮がスラッシングする場合（**Autocompact is thrashing** エラー）：

巨大なファイルや出力がコンテキストをすぐ埋めてしまう場合： 1. ファイルを行範囲で指定して読む（全体を一度に読まない） 2. **/compact keep only the plan and the diff** のようにフォーカスを指定 3. 大きなファイル処理はサブエージェントに委譲

6.4.3.7 フックが動作しない

症状: **PostToolUse** フックが実行されない

まず試すこと:

```
/doctor
```

インストール・設定・MCP・コンテキストの状態を自動チェックしてくれる。

ログを使ったデバッグ方法:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'hook triggered' >> /tmp/claude-hook
s.log"
          }
        ]
      }
    ]
  }
}
```

```
# ログを確認
tail -f /tmp/claude-hooks.log
```

--safe-mode で切り分け:

```
claude --safe-mode
```

プラグイン・MCP サーバー・フックを全て無効化した状態で起動します。これで問題が消えれば設定ファイルに原因があります。

6.4.3.8 API レート制限エラー

症状: **rate_limit_error** (429) や **529 Overloaded** が発生する

解決策: - より軽量なモデルを使う（`/model claude-haiku-4-5`） -
`maxTurns` を制限する - Anthropic Console でレート制限を確認・引き上げ
申請

6.4.3.9 Windows での文字化け

症状: 日本語が文字化けする、文字が豆腐（□）になる

VS Code / Cursor の統合ターミナルで発生する場合:

セッション内で以下を実行すると GPU レンダーが無効化される：

```
/terminal-setup
```

PowerShell のエンコーディング設定（一般的な文字化けの場合）：

```
$OutputEncoding = [System.Text.Encoding]::UTF8  
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
```

Windows での Path 区切り文字:

Claude Code は Windows でも基本的にスラッシュ（`/`）を使います。
PowerShell のコマンドではバックスラッシュ（`\`）が必要な場合があります。

6.4.3.10 WSL での検索パフォーマンス

症状: ファイル検索が遅い・結果が少ない

WSL でクロスファイルシステム（`/mnt/c/` 配下）で作業すると、ファイル読み込みのペナルティで検索結果が少なくなることがあります。

対処法: 1. **検索を絞り込む:** ディレクトリやファイル種別を指定する（「`auth-service`」パッケージで JWT 検証ロジックを検索して」） 2. **Linux ファイルシステムに移動:** プロジェクトを `/home/` 配下に置く 3. **ネイティブ Windows で実行:** WSL 経由でなく Windows 上で直接動かす

6.4.3.11 高 CPU / メモリ使用率とハング

症状: Claude Code が重い、または応答しない

対処法: 1. `/compact` でコンテキストを圧縮 2. 大きなビルドディレクトリを `.gitignore` に追加 3. `claude --safe-mode` で起動して設定の影響を切り分け 4. `/heapdump` でメモリスナップショットを取得（Linux/macOS はホームディレクトリに保存） 5. ハングした場合: `Ctrl+C` で中断。それでも応答しなければターミナルを閉じる

会話の再開（再起動後も続きから始められます）：

```
claude --resume
```

6.4.4 デバッグモード

詳細なログを出力するには：

```
# コマンドラインフラグ（推奨）
claude --debug

# 環境変数
DEBUG=1 claude
```

セッション起動後は `/debug` コマンドでも切り替え可能。

6.4.5 サポートリソース

| リソース | URL / コマンド |
|---------------|---|
| 公式ドキュメント | https://code.claude.com/docs/ |
| GitHub Issues | https://github.com/anthropics/claude-code/issues |
| ヘルプ | <code>/help</code> コマンドで確認 |
| フィードバック送信 | <code>/feedback</code> コマンドで直接報告 |
| 自動診断 | <code>claude doctor</code> / <code>/doctor</code> |

6.4.6 🏆 練習問題

1. **【確認】** `claude doctor`（または `/doctor`）で確認できる診断項目を3つ答えてください。また、Claude Code が起動しない場合と起動している場合でどちらのコマンドを使うべきか説明してください。
2. **【実践】** 意図的に `ANTHROPIC_API_KEY` を空にして Claude Code を起動し、表示されるエラーメッセージを確認しましょう（確認後は必ず元に戻すこと）。エラーメッセージにはどのような情報が含まれていましたか？
3. **【実践】** Claude Code が応答しなくなった場合の対処手順（`Ctrl+C` → ターミナル再起動 → `claude --resume`）を実際に試してみましょう。`--resume` で会話がどこまで復元されるか確認してください。
4. **【応用】** チームメンバーから「Claude Code が動かない」と言われた場合の診断フローを3ステップで説明してください。`/doctor`、エラーメッセージの確認、`--safe-mode` を活用した切り分けを含めてください。